

6 Structural Pattern Recognition

In the preceding chapters several methodologies of pattern classification and regression were presented, all based on a numerical similarity measure. Structural pattern recognition has a completely different approach, as briefly mentioned in section 1.4. Its main concern is the structure of a pattern, i.e., how a pattern can be described and interpreted as an organization of simple sub-patterns, usually designated as *pattern primitives*.

Structural pattern recognition has two main classes of methods: syntactic analysis and structural matching methods. Syntactic analysis methods are based on the use of formal language theory. Structural matching methods use specific techniques based on mathematical relations among the primitives.

Error bounds and generalization capability of structural recognition methods are problem-dependent. The performance of a syntactic analyser or of a structural matching method has to be assessed in a test set of patterns in the usual way.

Structural pattern recognition is often used in combination with statistical classification and/or neural network methods in hybrid approaches in order to achieve complex pattern recognition tasks, namely in applications of two-dimensional and three-dimensional object recognition.

6.1 Pattern Primitives

As the first step to describing and analysing pattern structures, they are decomposed into simple and well-defined elements, called *primitives*. The choice of primitives depends on the application, and can be of varied nature (see the bibliography section). In the following, we limit our discussion to methods of primitive extraction that can be applied to a large class of situations and are, therefore, quite popular.

6.1.1 Signal Primitives

Usually signals are decomposed into line segments or other simple curves. A *piecewise linear approximation* of a signal is by far the most popular method of signal decomposition. Suppose that we have a signal $s(x)$ that we want to approximate by a piecewise linear function $h(x)$ with d segments h_i . The approximation error is:

$$E = \sum_{i=1}^d \sum_{x_j \in h_i} \|s(x_j) - h_i(x_j)\|, \quad (6-1)$$

where an appropriate norm, usually the Chebychev norm or the Euclidian norm, is used to evaluate the deviations of $s(x_j)$ from $h_i(x_j)$.

A piecewise linear approximation of this kind is implemented in the *SigParse* program using the following simple algorithm:

1. Specify a maximum error, $E_{\max}$, for every line segment.
2. Start the approximation search with the first signal sample x_1 , which initiates the first line segment, $i=1$.
3. Set the number of signal samples to regress, $k=1$.
4. Generate a line regressing k signal samples, from x_i to x_{i+k-1} .
5. Evaluate E for the regressing line. If E is smaller than $E_{\max}$, increase k and go to 4, otherwise start a new line segment, increase i , and go to 3.

Figure 6.1 shows a piecewise linear approximation obtained with *SigParse* for an electrocardiographic (ECG) signal. The original signal has 702 samples. The approximation using the Chebychev norm with $E_{\max} = 17 \mu\text{V}$ (in a $682 \mu\text{V}$ pp. ECG) has only 21 segments.

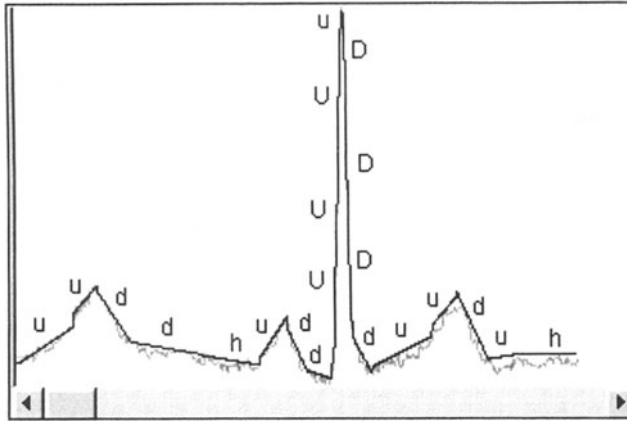


Figure 6.1. Piecewise linear approximation (black) of an ECG signal (grey). The line segments are labelled according to specified slope thresholds.

Sometimes it may be desirable to minimize the number of line segments by a careful adjustment of the segment ends, guaranteeing that an error norm is not exceeded, as described in Pavlidis (1973). In this case, the whole signal must be

known beforehand, whereas the previous algorithm can also be used in real time signal processing.

After the signal is decomposed into line segments, one can label these according to a specific rule. For instance, one may use the information on the slope of each segment to set slope thresholds in order to obtain the labels h , u , d , U , D mentioned in section 1.2.3. In that section we defined an h primitive as a line segment with zero slope; usually it is more convenient to set two slope thresholds, Δ_1 and Δ_2 , used for the segment labelling as u or d and as U or D , respectively, therefore defining h as any segment with slope below Δ_1 . This was done for the ECG signal shown in Figure 6.1 with $\Delta_1=0.1$ and $\Delta_2=5$. We will use this labelling approach in the following examples.

6.1.2 Image Primitives

Image primitives can be obtained through the application of a large variety of image analysis techniques, such as image segmentation, edge detection, contour following and medial axis transformation, described in detail in many textbooks on the subject (e.g., Duda and Hart, 1973 and Rosenfeld and Kak, 1982). In the following we describe some popular ways of deriving image primitives.

Chain Code and Templates

Chain code (*Freeman chain code*) constitutes an easy way of encoding a two-dimensional curve, represented in a rectangular grid with sufficient resolution. It consists of following the curve from a specified starting point and, for each line segment, connecting the grid points that fall closest to the curve. The grid point connections are then coded according to a set of octal primitives, as shown in Figure 6.2.

This technique is usually applied to binary images when one wants to encode the silhouette of an object by contour tracking, as shown in Figure 6.2, where the string starting at the highest vertical point and following the contour in the clockwise direction is: $x = 6007656454324221002$.

Chain coding of binary images is easy to implement and often provides a very good description. The difficulty with this technique is that, for an accurate enough representation, it may need a very fine grid, then generating very long strings. An alternative is to use template images, which are then considered the primitives. Classic templates are also shown in Figure 6.2. Starting from the highest vertical grid cell that is not empty, one can follow the contour in a clockwise direction, selecting the template that best corresponds with the current contour cell, obtaining in this case the string $x = 482611999\ 337741812$.

The use of templates for primitive description of binary images is applied in many image recognition tasks, for instance, in some implementations of character recognition and in fingerprint identification.

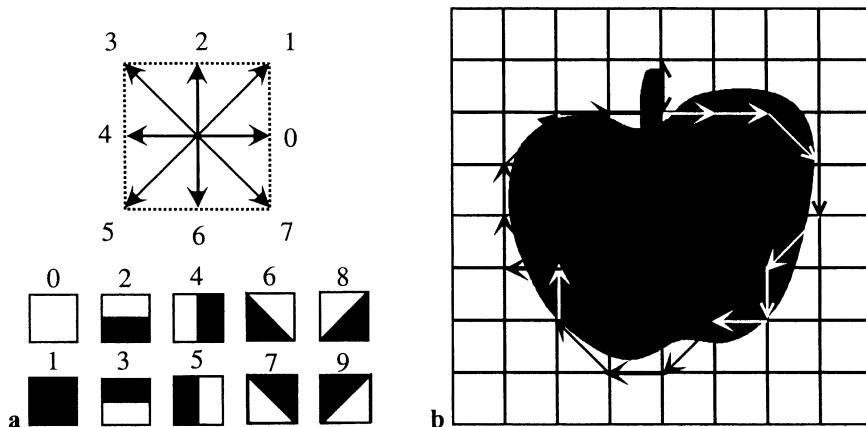


Figure 6.2. (a) Octal primitives (top) and templates (bottom); (b) Binary image with contour line segments according to the octal primitives.

Curve segments

Images described by curve segments, particularly line segments, can be obtained by applying edge detectors to them, as shown in Figure 6.3. They are usually called *shape primitives* or *image silhouette primitives*. In some applications, instead of image silhouettes, one may be interested in obtaining a skeletonised description of an image, by applying thinning or medial axis transformation to a binary image. An application of this technique can be found in character recognition (see e.g., Di Baja, 1996).

After obtaining the curve segment description, one may label the segments in a similar way to what we have done for line primitives of signals, or extract other descriptions from the curve segments (perimeter, curvature, slope of the segment linking the ends, etc.).

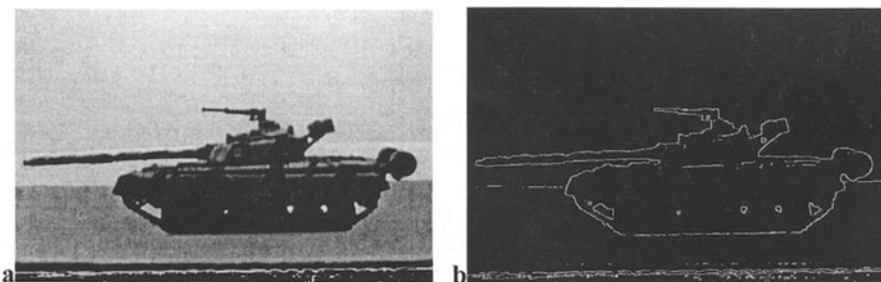


Figure 6.3. (a) Image of a tank (toy model); (b) Image contours detected by an edge detection algorithm.

A particularly useful description can be obtained from line segments derived by the application of the *Hough transform*, a transformation that maps a straight line $y = ax + b = \rho \cos \theta$ into a point in the (a, b) plane or the (ρ, θ) plane. The straight line Hough transform can also be applied to the description of arbitrary curves. Details on how to use this technique can be found in Pao and Li (1992).

Regions

The segmentation of an image into distinct regions is often used as a means of obtaining structural descriptions of images, e.g., based on proximity relations of region centroids and using labels related to the image properties of the regions such as perimeter, area, colour, light intensity and texture. This approach is used, for instance, in image registration applications, such as the structural matching of satellite images (see e.g., Ton and Jain, 1989), which we discuss in section 6.4.2.

Region primitives such as centroids and corners can also be obtained from binary images by using morphologic operators (see e.g., Shapiro, 1988).

6.2 Structural Representations

The representation of a pattern in terms of its constituent elements can be done in many ways. Here we will describe the most common ways of structural representation.

6.2.1 Strings

A *string* is an ordered sequence of symbols, each symbol representing a primitive. We denote by S the set of all possible strings that can be built with the elements of a symbol set T . A string $\mathbf{x}$ is then a sequence of symbols of T represented as:

$$\mathbf{x} = a_1 a_2 \dots a_m, \quad a_i \in T. \quad (6-2)$$

The number of symbols, m , is the *string length*, denoted $|\mathbf{x}|$. The string with no symbols, $m=0$, is called the *null string* and denoted λ .

We define the *concatenation* of strings $\mathbf{x} = a_1 a_2 \dots a_m$ and $\mathbf{y} = b_1 b_2 \dots b_n$ with m symbols and n symbols, respectively, and denote this operation as $\mathbf{x} + \mathbf{y}$, yielding the string with $m+n$ symbols:

$$\mathbf{z} = \mathbf{x} + \mathbf{y} = a_1 a_2 \dots a_m b_1 b_2 \dots b_n. \quad (6-3)$$

Note that the null string is the neutral element for the concatenation:

$$\mathbf{x} + \lambda = \lambda + \mathbf{x} = \mathbf{x}. \quad (6-3a)$$

Strings are useful for representing concatenated structures, for instance, segment joining in signal descriptions. Even in some apparently complex relations of primitives, describing 2D and 3D structures, it is still possible to use strings, as we will see in section 6.3.4.

Sometimes the ordered sequence of primitives expressed by a string does not contain enough information to achieve pattern discrimination. For instance, when analysing signals, such as electrocardiograms or heart rate tracings, not all peaks described by strings such as $\mathbf{x} = ud$ or $\mathbf{x} = du$ are relevant, i.e., meet the requirements necessary for attributing them to a certain class of waves. Extra conditions often have to be imposed on the duration and amplitude of the peaks. We deal with this requirement through the formalism of *attributed strings*, i.e., strings that have an associated *attribute vector*:

$$\mathbf{a}(\mathbf{x}) = [a_1 \ a_2 \ \dots \ a_k]' \in \mathfrak{R}^k. \quad (6-4)$$

For waveforms described by line segments the attribute vector usually has two elements, namely duration and height of each segment. In some cases there are categorical attributes (e.g., colour), and the attribute vector will contain labels (e.g., red). In the case of strings with single labels, we will refer to them as *labelled strings*.

6.2.2 Graphs

A graph G is an ordered pair:

$$G = \{N, R\}, \quad (6-5)$$

where N , the *node set*, is a set of nodes, and R , the *edge set*, is a set of binary relations defined in $N \times N$.

The elements of R represent arcs (or edges) connecting nodes of N . An arc of G is denoted (a, b) with $a, b \in N$.

A directed graph, or *digraph*, is a graph where each arc has a direction, emanating from a node and incident onto either another node or the same node. An arc from a digraph, represented by the pair (a, b) , means that a is the emanating node and b is the incident node. Generally, in a digraph, (a, b) will mean a different relation than (b, a) .

The number of nodes b , such that $(a, b) \in G$, i.e., the number of emanating arcs from a , is the *out-degree* of node a . The number of nodes a , such that $(a, b) \in G$, i.e., the number of arcs incident in b , is the *in-degree* of node b .

Graphs are a very flexible way of expressing relations among primitives. As there are attributed and labelled strings, we can similarly have attributed or labelled graphs. Figure 6.4 illustrates the use of digraphs in character recognition. At the left side of the figure, primitives and relations describing capital letters are indicated. At the right side of the figure, two examples are shown of digraphs using these primitives and relations, for the letters "R" and "E".

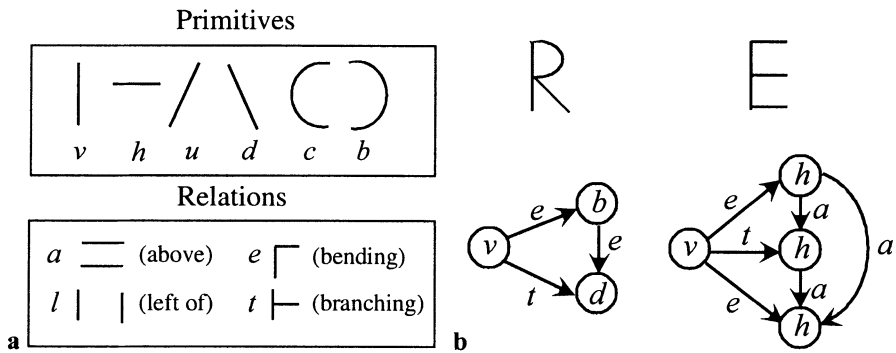


Figure 6.4. Primitives and relations used to describe capital letters (a) and labelled digraphs for the letters R and E (b).

6.2.3 Trees

A tree is an undirected graph with no closed loops (*acyclic graph*) and with a special node, the *root node*, with in-degree of zero and every other node with out-degree ≥ 1 , except the terminal or *leaf nodes*, which have out-degree zero.

Trees provide a useful representation of hierarchical structures, namely when such structures have a recursive nature. Notice that we have already used trees for hierarchical classifiers in previous chapters.

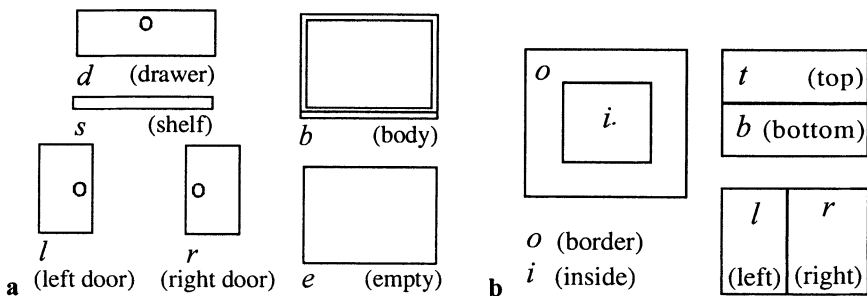


Figure 6.5. Primitives (a) and relations (b) used to describe furniture, as in Figure 6.6.

As an illustration of trees used for the description of hierarchical structures, we consider the task of recognizing pieces of furniture, based on the set of primitives and the set of relations shown in Figure 6.5. Each relation is applied hierarchically. For instance, we can apply the relations "left"- "right" to any sub-pattern previously labelled as "bottom". Figure 6.6 shows how a tree using these primitives and

relations can represent a cupboard. This method has been used in several applications, namely in character recognition (e.g., Tai and Liu, 1990; Chen and Lee, 1996).

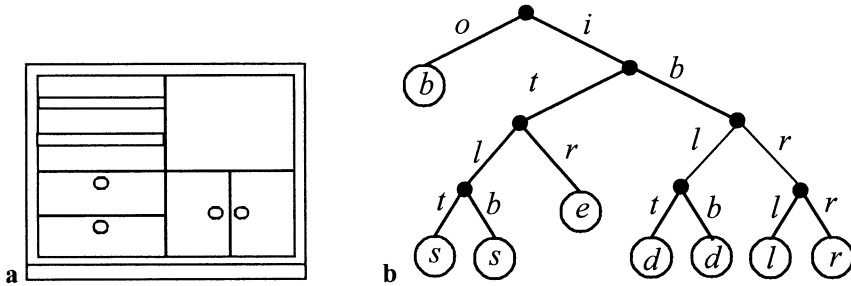


Figure 6.6. A cupboard (a) and its description tree (b) using the primitives and relations of Figure 6.5.

6.3 Syntactic Analysis

Syntactic analysis is the approach to structural pattern recognition based on the theory of formal languages. In the literature, this approach is generally called *Syntactic Pattern Recognition*. The use of grammars to describe patterns is advantageous, given the well-developed body of knowledge in this area and the standardized way one can apply the same approach in many different circumstances.

The disadvantages of the syntactic approach relate to the lack of sufficient representational capability for complex patterns, and also the difficulty in automating inference and learning procedures, which means that in general one first has to devise the appropriate grammar for the problem at hand.

6.3.1 String Grammars

The theory of formal languages allows us to apply structure rules to strings of pattern primitives in a similar way as we apply grammar rules to words in a natural language.

We proceed to define a formal string grammar as a quadruple $G = (T, N, P, S)$:

1. T is a set of symbols, called *terminal symbols*, corresponding in the case of the patterns to the set of primitives, also called *pattern alphabet*.

The set of strings built with the elements of T is usually denoted T^+ :

$$T^+ = \{a_1 a_2 \cdots a_n; a_i \in T, n \geq 1\}. \quad (6-6a)$$

A formal language L is a subset of $T^+ \cup \{\lambda\}$, constituted by strings obeying certain rules.

2. N is a set of *class symbols*, also called *non-terminal symbols*, i.e., symbols that denote classes of elements of T . For instance, when describing natural languages, we would use non-terminal symbols to denote the classes of words that are "nouns", "adjectives", "verbs", etc.

The sets T and N are disjoint and their union, $V = T \cup N$, constitutes the language vocabulary.

3. P is a set of syntactic rules, known as *production rules*, used to generate the strings. Each rule is represented as:

$$\alpha \mapsto \beta, \quad \text{with} \quad \alpha \in V^+, \beta \in V^+ \cup \{\lambda\}. \quad (6-6b)$$

The production rule $\alpha \mapsto \beta$, read " α produces β ", means that any occurrence of α in a string can be substituted by β .

4. S is a special *parent class symbol*, also called *starting symbol*, used to start the generation of any string with the rules of P .

Let us apply the above definitions to a waveform recognition example, by considering signal waveform descriptions where a waveform is built with only three primitives: horizontal segments, h ; upward segments, u ; downward segments, d . Hence, the pattern alphabet is, in this case, $T = \{h, u, d\}$.

We now consider the following classes of strings of T :

P^+ : upward peak;
 P^- : downward peak;
 H : horizontal plateau.

Hence, $N = \{P^+, P^-, H\}$.

We can now define a production rule for an upward peak as:

$$P^+ \mapsto ud. \quad (6-7a)$$

Another production rule for the same class could be:

$$P^+ \mapsto uP^+d. \quad (6-7b)$$

As this last rule is a recursive rule, we are in fact describing upward peaks as an arbitrary number of u primitives followed by the same number of d primitives. We can write the previous rules (6-7a) and (6-7b) compactly as:

$$P^+ \mapsto u(P^+ | \lambda) d. \quad (6-7c)$$

The symbol $|$ denotes an exclusive "or" choice and the parentheses allow a kind of factorisation, meaning that P^+ can be substituted with uP^+d or with $u\lambda d$, which is in fact ud .

Consider now that our waveform grammar has the following set of productions:

$$P = \left\{ \begin{array}{l} S \mapsto (P^+ | P^- | H)(S | \lambda), \quad P^+ \mapsto u(P^+ | \lambda) d, \\ P^- \mapsto d(P^- | \lambda) u, \quad H \mapsto h(H | \lambda) \end{array} \right\}. \quad (6-7d)$$

The starting symbol S stands for "any waveform obtainable with the rules of P ".

The set P determines a language $L(G)$ (or simply L) of waveforms, which is a subset of all possible waveforms of $T^+ \cup \{\lambda\}$. Consider the waveform of Figure 6.7a, represented by the pattern string $\mathbf{x} = hhuuddh$. Is it an element of L (describable by P)? In order to answer this question, we have to see if it is possible to generate $\mathbf{x}$ using the rules of P starting from S :

$$S \mapsto HS \mapsto HP^+S \mapsto HP^+H \mapsto hHuP^+dh \mapsto hhuuddh. \quad (6-7e)$$

We have indeed been able to generate $\mathbf{x}$ with the rules of P , or, in the jargon of formal language theory, we were able to *parse* $\mathbf{x}$.

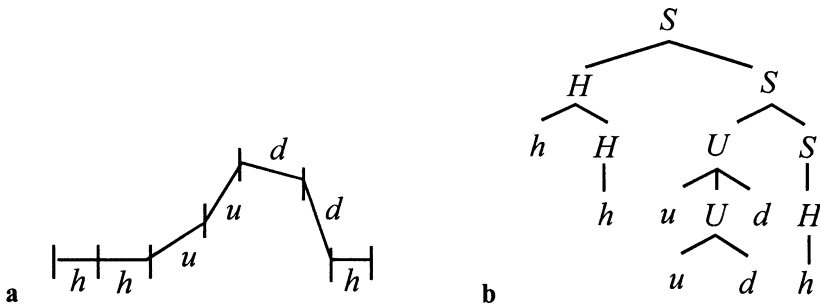


Figure 6.7. A signal wave (a) represented by the string $\mathbf{x} = hhuuddh$ and its top-down parsing tree (b).

The chain of rules (6-7e) generating $\mathbf{x}$ is called a *derivation* of $\mathbf{x}$. The application of rules for obtaining this derivation is called *top-down parsing*, since we started from the initial symbol S and went down until the terminal symbols, as shown in the *parsing tree* of Figure 6.7b. It is also possible to start from the string applying the rules in reverse order as a *bottom-up parsing* chain:

$$hhuuddh \mapsto hHuP^+dh \mapsto HP^+H \mapsto HP^+S \mapsto HS \mapsto S. \quad (6-7f)$$

Since we were able to reach the starting symbol, we have confirmed that $\mathbf{x}$ belongs to L generated by P , i.e., the grammar G recognizes $\mathbf{x}$, or $\mathbf{x} \in L(G)$.

Let us now consider the pattern string $\mathbf{y} = hhuudh$. It is easy to see that $\mathbf{y}$ does not belong to $L(G)$, since no parsing is possible in this case. We see, therefore, that the task of determining whether or not a pattern belongs to a given class is reduced to seeing whether or not it can be parsed with the grammar rules. The reader may find it interesting to change the rule set P in order to recognize string $\mathbf{y}$, and other strings as well.

Let us now modify the rules of P^+ and P^- as follows:

$$P = \left\{ \begin{array}{l} S \mapsto (P^+ \mid P^- \mid H)(S \mid \lambda), \quad P^+ \mapsto u(S \mid \lambda)d, \\ P^- \mapsto d(S \mid \lambda)u, \quad H \mapsto h(H \mid \lambda) \end{array} \right\}. \quad (6-7g)$$

Consider the string $\mathbf{x} = udud$. It can be derived in two ways:

$$S \mapsto P^+S \mapsto udS \mapsto udP^+ \mapsto udud. \quad (6-7h)$$

$$S \mapsto P^+ \mapsto uSd \mapsto uP^-d \mapsto udud. \quad (6-7i)$$

When a grammar admits more than one derivation for the same string we call it an *ambiguous* grammar. In general, there is a different semantic value for each derivation. In the present example, we may interpret $udud$ as a sequence of two positive peaks or as a positive peak with a negative peak in the middle.

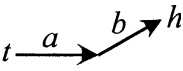
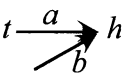
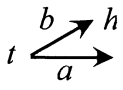
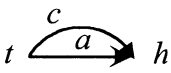
String grammars can be generalized to apply to more complex structures such as trees and graphs.

6.3.2 Picture Description Language

Grammar strings that use concatenation as the only operation have a limited representational capability, namely when relations among two-dimensional or three-dimensional objects have to be represented. One way to cope with this limitation is to use higher dimensional grammars such as tree or graph grammars. This approach has met limited success in practical applications, given the inherent difficulty of dealing with such grammars, namely in what concerns the building of efficient parsers. Another way is to incorporate operations that are more complex than concatenation into the grammar. This is the approach of the *picture description language*, PDL, which provides an adequate representation of two-dimensional patterns.

In PDL each primitive has two attaching points, *tail* and *head*, where it can be linked to other primitives. The set of terminals includes four binary operators and one unary operator that can be used to join primitives as shown in Table 6.1.

Table 6.1. PDL operators and their meaning.

Operator	Meaning	Geometric interpretation
$a+b$	$head(a+b) = head(b)$ $tail(a+b) = tail(a)$	
$a-b$	$head(a)$ linked to $head(b)$ $head(a-b) = head(b)$ $tail(a-b) = tail(a)$	
$a \times b$	$tail(a)$ linked to $tail(b)$ $head(a \times b) = head(b)$ $tail(a \times b) = tail(b)$	
$a * b$	$head(a)$ linked to $head(b)$ and $tail(a)$ linked to $tail(b)$ $head(a * b) = head(a)$ $tail(a * b) = tail(a)$	
$\sim a$	$head(\sim a) = tail(a)$ $tail(\sim a) = head(a)$	

It can be shown that any digraph with labelled edges can be represented by a PDL expression. Therefore, any pattern that can be represented by a digraph can also be described by a PDL expression. As an example, let us look back to the digraphs of Figure 6.4 and describe them with PDL strings, denoting **r** for the letter "R" and **e** for the letter "E", respectively, using the new version of the primitives shown in Figure 6.8:

$$\mathbf{r} = (v + (\sim b)) + (\sim d); \quad (6-8a)$$

$$\mathbf{e} = (v \times h) + ((v \times h) + h). \quad (6-8b)$$

Note that the strings now incorporate the PDL operator symbols and the parentheses as terminals. Note also that, in general, more than one representation is possible. For instance, the following are also valid expressions:

$$\mathbf{r} = d + (b + (\sim v)); \quad (6-9a)$$

$$\mathbf{e} = (\sim h) + ((v - (\sim h)) + (v - (\sim h))). \quad (6-9b)$$

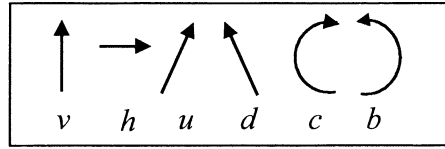


Figure 6.8. Primitives for character description using picture description language.

String grammars are easily generalized to PDL descriptions. We will show this by presenting the string grammar that describes the characters "P", "R", "F" and "E":

$$\begin{aligned}
 G &= \{T, N, P, S\}; \\
 T &= \{v, h, d, b, \sim, +, -, (,)\}; \\
 N &= \{S, P, R, F, E, \text{LEFTCORNER}\}; \\
 P &= \{ S \mapsto (P|R|F|E), \quad R \mapsto d + P, \quad P \mapsto b + (\sim v), \\
 &\quad \text{LEFTCORNER} \mapsto v - (\sim h), \quad F \mapsto \text{LEFTCORNER} + \text{LEFTCORNER} \\
 &\quad E \mapsto (\sim h) + F \}.
 \end{aligned}$$

Shape descriptions can also be generated using other types of features, namely line segments derived by chain coding, as explained in section 6.1.2. A variant of this method, using appropriate concatenation rules, is described in the work of Nishida (1996) and applied to structural analysis of curve deformation.

6.3.3 Grammar Types

The string grammar defined in section 6.3.1 states the production rules in the most general way, constituting what is called an *unrestricted grammar*. By imposing restrictions on the production rules of this grammar, it is possible to define other types of grammars, as listed in Table 6.2.

Table 6.2. Chomsky's hierarchy of grammar types.

Type	Name	Production rules
G_0	Unrestricted	$\alpha \mapsto \beta$, with $\alpha \in V^+$, $\beta \in V^+ \cup \{\lambda\}$
G_1	Context-sensitive	$\alpha \mapsto \beta$, with $\alpha, \beta \in V^+$ and $ \alpha \leq \beta $
G_2	Context-free	$\alpha \mapsto \beta$, with $\alpha \in N$, $\beta \in V^+$
G_3	Regular	$\alpha \mapsto a\beta \mid a\lambda$, with $\alpha, \beta \in N$, $a \in T$

The following relations exist for these four types of grammars: $G_0 \supset G_1 \supset G_2 \supset G_3$. This is called the Chomsky' hierarchy, named after Noam Chomsky whose contribution to the formal language theory was capital.

The representational power of a grammar decreases as we go from a type G_0 grammar to a type G_3 grammar.

Production rules of a type G_1 grammar can also be written:

$$\alpha_1 \alpha_2 \mapsto \alpha_1 \beta \alpha_2, \quad \text{with } \alpha \in N, \quad \beta \in V^+, \quad \alpha_1, \alpha_2 \in V^+ \cup \{\lambda\}. \quad (6-10)$$

Therefore, α substitutes β in the context of α_1 and α_2 . This justifies the context-sensitive name given to this type of grammars (see also Exercise 6.2). For a context-free grammar, the substitution of α by β is independent of the context in which α appears. Since these grammars allow self-embedded productions of the type $\alpha \mapsto \beta_1 \beta \beta_2$, they can generate arbitrarily long sequences of terminals by iteration of the same rule. Regular grammars are a special case of context-free grammars that allow very simple representation and parsing, as will be explained in the following section.

As we saw in the previous section, the task of assessing a pattern structural class is equivalent to seeing whether it can be parsed by a suitable parser, which applies the grammar's rules. Parser building for G_0 and G_1 grammars can be a very hard task. Fortunately, structural recognition problems can often be solved using G_2 and G_3 grammars, which allow a broad range of string expressiveness. For these grammars there is a bulk of knowledge about parser building. As a matter of fact, there are commercialised and free-ware parsers that can, in principle, be applied to syntactic pattern recognition. The reader may also obtain information on how to build a parser for G_2 grammars in (Schalkoff, 1992).

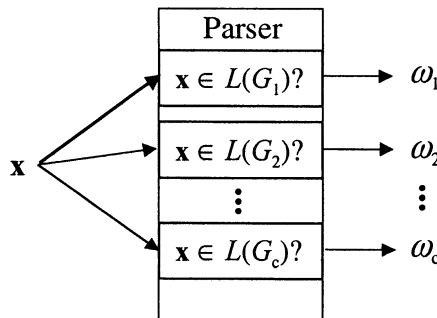


Figure 6.9. Block diagram of a syntactic pattern classifier. $L(G_i)$ are the class languages.

Figure 6.9 represents the block diagram of a syntactic recogniser, having the primitive string as input and class labels together with the syntactic derivations obtained by the parser as outputs. A language $L(G_i)$ represents the set of all strings corresponding to class ω_i . For the example in the previous section the languages would correspond to the sub-grammars producing the derivations for the characters "P", "R", "F" and "E".

Notice that, when parsing a string, one of the following three situations may occur:

- The string belongs to only one of the class languages. This is the desirable situation.
- The string belongs to more than one of the class languages. This is an ambiguous "borderline" string, reminiscent of feature vectors that lie in a reject region.
- The string does not belong to any of the class languages. This is also a reject or a "don't care" situation. For instance, when parsing strings of signal primitives, we may encounter in the tail of a parsed signal wave a sequence of primitives that are not parsed, corresponding to a "don't care" situation.

6.3.4 Finite-State Automata

A broad variety of syntactic recognition tasks can be achieved with the simple $G3$ grammars referred to previously. Fortunately for these grammars the parsing task is quite simple to implement, using a device known as a *finite-state automaton*.

We define a finite-state automaton (or *finite-state machine*) as a quintuple $M = \{T, Q, \delta, q_0, F\}$:

- T is a symbol set (corresponding to the set of primitives);
- Q is a set of states of the machine;
- δ is a function mapping from $Q \times T$ into subsets of Q ;
- $q_0 \in Q$ is the initial machine state;
- $F \subseteq Q$ is the set of final states.

A finite-state automaton can be represented as a labelled digraph, known as *state-diagram*, with arcs representing state transitions. Let us exemplify this with:

$$T = \{h, u, d\}$$

$$Q = \{S, U+, D+, U-, D-, F\}; \quad q_0 \equiv S$$

$$\delta(S, h) = S; \quad \delta(S, u) = U+;$$

$$\delta(S, d) = D-;$$

$$\delta(U+, h) = S; \quad \delta(U+, u) = U+;$$

$$\delta(U+, d) = D+;$$

$$\delta(D+, h) = F; \quad \delta(D+, u) = F;$$

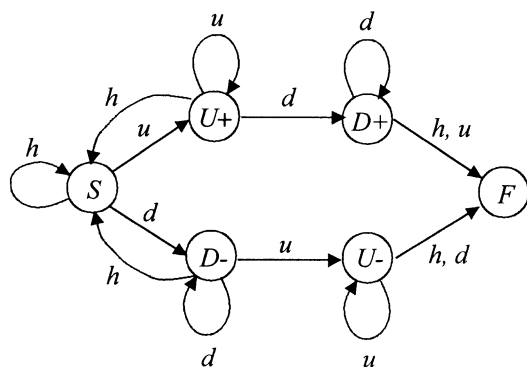
$$\delta(D+, d) = D+;$$

$$\delta(D-, h) = S; \quad \delta(D-, u) = U-;$$

$$\delta(D-, d) = D-;$$

$$\delta(U-, h) = F; \quad \delta(U-, u) = U-;$$

$$\delta(U-, d) = F;$$



Present State	Input Symbol	Next State
S	h	S
S	u	U+
S	d	D-
U+	h	S
U+	u	U+
U+	d	D+
D+	h	F
D+	u	F
D+	d	D+
D-	h	S
D-	u	U-
D-	d	D-
U-	h	F
U-	u	U-
U-	d	F

Figure 6.10. Finite-state automaton represented by its state-diagram, with corresponding state-transition table. The automaton recognizes waveform peaks without horizontal plateaus.

Figure 6.10 shows the state-diagram corresponding to this finite-state automaton, together with the *state-transition table*, which is a convenient way of representing the mapping δ .

Assuming that the symbols of T represent the signal primitives mentioned already in section 6.3.1, it is clear that this finite-state automaton recognizes any positive or negative peaks that do not include any horizontal plateaus.

A string is said to be *accepted* by M if starting with the initial state we can use the successive string symbols to progress in the state-diagram until a final state. The set of all strings *accepted* by M , denoted $A(M)$, is formally defined as:

$$A(M) = \{x; \delta(q_0, x) \in F\}. \quad (6-11)$$

For any regular grammar $G = \{T, N, P, S\}$, it can be proved that there is a finite-state automaton M such that $A(M) = L(G)$ and vice-versa. The finite-state automaton M uses the set of terminals T , and is implemented as follows:

- $Q = N \cup \{A\}$;
- $q_0 = S$;
- A production $\alpha \mapsto a\beta$ corresponds to $\delta(\alpha, a) = \beta$;
- A production $\alpha \mapsto a$ corresponds to $\delta(\alpha, a) = F$;
- If P contains the production $S \mapsto \lambda$ then $F = \{S, A\}$ else $F = \{A\}$.

Specifying a grammar that corresponds to a finite-state machine is now an easy task, by simply reversing the above rules.

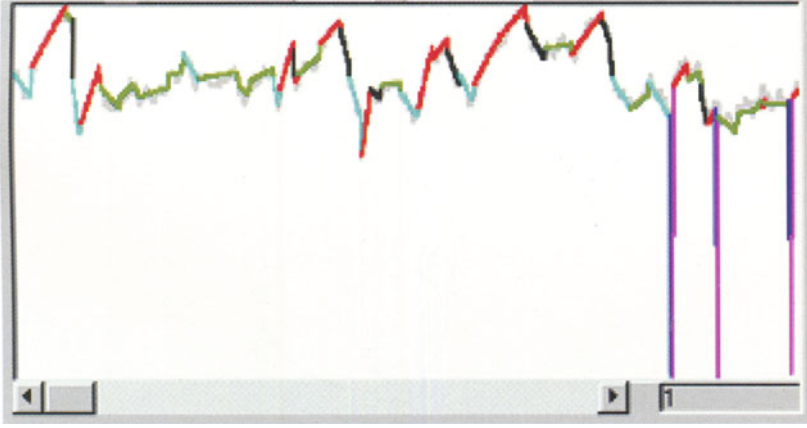


Figure 6.11. FHR signal parsing after approximation by line segments. Line segments are labelled as: h =green; u =red; d =cyan; U =magenta; D =blue. Final states are represented in black. The original signal (in the background) is in grey.

We see therefore that a finite-state machine constitutes a parser for a regular grammar. As a matter of fact, given the expressiveness of a state-diagram, one usually goes directly to the specification of the state-diagram or the state-transition table, omitting the specification of the grammar.

The *SigParse* program allows one to specify a state-transition table for recognizing regular string grammars. We exemplify its use by considering foetal heart rate (FHR) signals, such as the one depicted in Figure 6.11.

Imagine that we want to detect the presence of downward spikes (class *DSPIKE*) in such signals. We start by performing a line segment approximation using the Chebychev norm with a tolerance $E_{\max}=3$, and use slope thresholds of 0.3 and 20 for the segment labelling. These are colour-coded as shown in Figure 6.11. Next, we use the following simple set of rules for the recognition of *DSPIKE* strings:

$$\{DSPIKE \mapsto DA; \quad A \mapsto hB; \quad A \mapsto U; \quad B \mapsto U\}. \quad (6-12a)$$

For the detection of accelerations (class *ACEL*), the set of rules is somewhat more elaborate:

$$\{ACEL \mapsto uA; \quad A \mapsto d; \quad A \mapsto uA; \quad A \mapsto hB; \quad B \mapsto uA; \quad B \mapsto d\}. \quad (6-12b)$$

This corresponds to the finite-state machine of Figure 6.12.

Events detected as "accelerations" by this machine may not qualify as true accelerations in the clinical sense. They only qualify as accelerations if they satisfy a set of conditions on the minimum duration and minimum amplitude. For

instance, in Figure 6.11 the second and fourth events do not satisfy such conditions, although they parse as "accelerations". We will see in the following how to take into account primitive attributes.

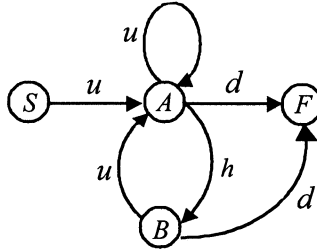


Figure 6.12. State-diagram of a finite-state automaton for the recognition of FHR accelerations.

6.3.5 Attributed Grammars

Until now, we have only considered grammars for string descriptions alone. However, as seen in the previous example of FHR acceleration detection, the qualitative information provided by a string may not be enough for correct recognition. In the mentioned example, if we want to detect "true" accelerations, we will have to appropriately use the information on the length and slope of the segments. In other applications other information may have to be used, such as angle orientation of line segments, textural information or surface orientation. For this purpose, we will have to deal with attributed strings and use an *attributed grammar*, with the same definition as before, plus the addition of *semantic rules*. A pair "syntactic rule - semantic rule" for a regular grammar is:

$$\alpha \mapsto b; \quad \mathbf{a}(\alpha) = f(\mathbf{a}(b)), \quad (6-13)$$

where $\mathbf{a}(\alpha)$ is the attribute vector of the non-terminal α computed in terms of the attribute vector of the terminal b .

An attribute vector can be interpreted as a feature vector. When parsing a string with attributed grammars, one may assume that the production rules for each symbol make its attribute dependent either on the left or on the right side of the productions. For instance, in top-down parsing, attribute values for each symbol are inherited from each right side symbol. We exemplify this for the first production rule of the FHR acceleration rules (6-12b):

$$l(ACEL) = l(u) + l(A); \quad t(ACEL) = t(u) + t(A), \quad (6-14)$$

where l is the length attribute and t is height of the upper segment end.

The height attribute is computed from the length and slope for the u primitives, and may be set to zero for h and d . When a wave event is correctly parsed as an acceleration, the values of $l(ACEL)$ and $t(ACEL)$ are compared with pre-specified thresholds in order to arrive at a final decision.

Attributed strings can be used for shape categorization of object silhouettes described by line segments, using as attributes the length and the angle of the segments. Details on this application can be found in the works of You and Fu (1979) and Stenstrom (1988).

6.3.6 Stochastic Grammars

Stochastic grammars provide a good example of combining structural and statistical approaches and applying them to pattern recognition problems. Stochastic grammars are particularly useful in situations where the same pattern can be interpreted as belonging to more than one class, therefore having the possibility of being generated by alternative grammars.

A *stochastic grammar* is a grammar whose production rules have probabilities associated with them, such that for every symbol α_i producing symbols β_j there are probabilities P_{ij} satisfying:

$$P_{ij} : \alpha_i \mapsto \beta_j \quad (\text{or } \alpha_i \mapsto \beta_j), \quad \text{with} \quad 0 < P_{ij} \leq 1, \quad \sum_{j=1}^{n_i} P_{ij} = 1. \quad (6-15)$$

Therefore, the probabilities associated with the same left side symbol of a production add up to one.

The probability of $\mathbf{x} \in L(G)$, $P(\mathbf{x} | G)$, is computed as:

1. If $\mathbf{x} \in L(G)$ is unambiguous and has a derivation with k rules, each with probability P_j , then:

$$P(\mathbf{x} | G) = \prod_{j=1}^k P_j. \quad (6-16a)$$

2. If $\mathbf{x} \in L(G)$ is ambiguous and has l different derivations with probabilities $P_i(\mathbf{x} | G)$, then:

$$P(\mathbf{x} | G) = \sum_{i=1}^l P_i(\mathbf{x} | G). \quad (6-16b)$$

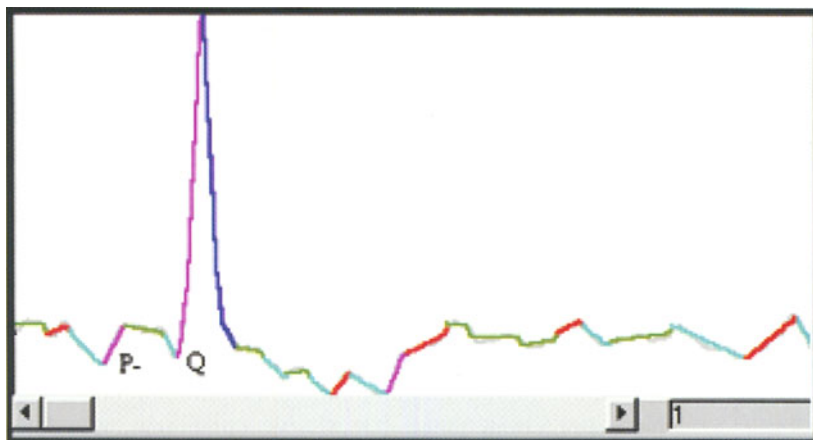


Figure 6.13. An ECG signal, zoomed 4x, described by a piecewise approximation using Chebychev norm with tolerance 20, with segments labelled using slope thresholds of 5 and 22. Colour coding as in Figure 6.11.

As an example, let us consider the distinction between negative P waves and Q waves in an electrocardiographic signal, as shown in Figure 6.13. A brief description of these waves can be found in section 1.3. P waves can be positive and/or negative. Q waves, by definition, are negative. Figure 6.13 shows a negative P wave, described by the string dU , followed by a Q wave, described by the same string.

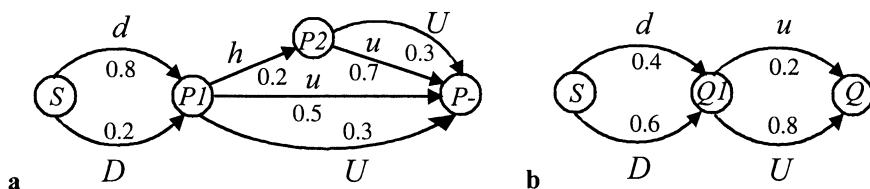


Figure 6.14. State-diagrams of finite-state automata for the recognition of negative P waves (a) and Q waves (b).

The state-diagrams of finite-state automata recognizing negative P waves and Q waves are shown in Figure 6.14. Each arc in these diagrams is labelled with the corresponding transition probability. Notice that rule (6-15) is satisfied. Notice also that the state transition probabilities reflect the fact that a negative P wave is usually less peaked than a Q wave.

The string dU , corresponding to either the first negative P wave or the following Q wave in Figure 6.13, can be parsed by both grammars corresponding to the state-diagrams of Figure 6.14.

The probabilities of generating this string are:

$$P\text{- grammar: } P(\mathbf{x} \mid P-) = 0.8 \times 0.3 = 0.24;$$

$$Q \text{ grammar: } P(\mathbf{x} \mid Q) = 0.4 \times 0.8 = 0.32.$$

When a string can be accepted by multiple grammars we apply the Bayes rule, used already in statistical classification, in order to arrive at a decision. For this purpose, assume that $\mathbf{x}$ can be produced by c grammars G_i with probabilities $P(\mathbf{x} \mid G_i)$. Assume further, that the prior probabilities for the productions are $P(G_i)$. Application of the Bayes rule corresponds to choosing the grammar that maximizes $P(G_i \mid \mathbf{x})$, or equivalently:

$$\text{Decide } \mathbf{x} \in L(G_k) \text{ if } P(G_k) P(\mathbf{x} \mid G_k) = \max \{P(G_i) P(\mathbf{x} \mid G_i), i=1, \dots, c\}. \quad (6-17)$$

In the case of the electrocardiographic signals, the distinction between negative P waves and Q waves is usually a simple task, based on the presence of a peaked R wave after the Q wave. However, there exist less frequent situations where Q waves can appear isolated and there are also rare rhythmic pathologies, characterized by multiple P waves, which may superimpose over the QRS sequence. The signal shown in Figure 6.13 corresponds precisely to such a pathology. Supposing that in this case the prior probabilities are:

$$P(P-) = 0.6; P(Q) = 0.4,$$

we obtain:

$$P\text{- grammar: } P(P-)P(\mathbf{x} \mid P-) = 0.144;$$

$$Q \text{ grammar: } P(Q)P(\mathbf{x} \mid Q) = 0.128;$$

and would decide then for the presence of a negative P wave, although the distinction is somewhat borderline.

In the case of "typical" negative P and Q waves there is a clear distinction:

Typical negative P wave, with string du :

$$P\text{- grammar: } P(P-)P(\mathbf{x} \mid P-) = 0.336;$$

$$Q \text{ grammar: } P(Q)P(\mathbf{x} \mid Q) = 0.032.$$

Typical Q wave, with string DU :

$$P\text{- grammar: } P(P-)P(\mathbf{x} \mid P-) = 0.036;$$

$$Q \text{ grammar: } P(Q)P(\mathbf{x} \mid Q) = 0.192.$$

For the situation shown in Figure 6.13, with string dU , one would probably consider the application of further criteria for discriminating negative P waves from Q waves before making any definite decision.

6.3.7 Grammatical Inference

Until now the grammar rules were supposed to be known beforehand. Grammatical inference is the learning process that extracts from a training set of classified examples the grammatical rules that are able to syntactically describe those examples. It is therefore a concept-driven approach, which bears some resemblance to the supervised classification methods. Instead of parameter learning we are here interested in rule learning.

Many algorithms have been proposed for grammatical inference; a good survey on these is given in (Miclet, 1990). Particular simple and efficient methods have been devised for inferring regular grammars. One of these methods, called the *successor method*, starts by building a state diagram with a node corresponding to each different production symbol, and then merges the nodes that have the same successors. Let us imagine that we were given the following set of examples of FHR accelerations: $T = \{ud, uhd, uud, uuhuud, uhud\}$. From these examples we start listing all the successors of each symbol in T :

- State S – Successors of λ in T : $\{u\}$
- State q_u – Successors of u in T : $\{u, h, d\}$
- State q_h – Successors of h in T : $\{u, d\}$
- State q_d – Successors of d in T : $\emptyset$

With this information we can now build the finite-state automaton of Figure 6.15, where each node corresponds to a symbol, and the arcs are labelled in correspondence with the successors of the symbol. As a matter of fact, the automaton of Figure 6.15 is the same as the one shown in Figure 6.12, with $q_d = F$.

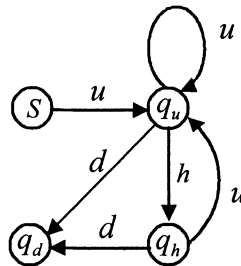


Figure 6.15. Finite-state diagram of an example-inferred automaton for FHR accelerations.

Imagine now that the training set T also contained the example $uhhud$. In this case, the successors of h contain h itself, therefore, the state diagram shown in Figure 6.16a represents the corresponding automaton.

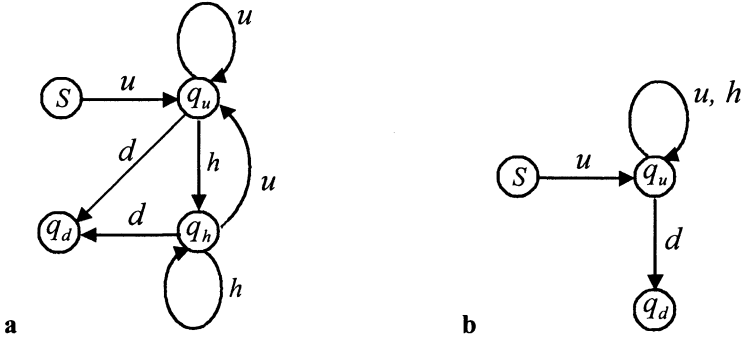


Figure 6.16. A modified finite-state diagram for FHR accelerations (a) and its minimized version (b).

Since states q_u and q_h now have the same successors, they can be merged, leading to the minimized automaton shown in Figure 6.16b. The rules for building the initial state diagram based on successors of symbols and the rules for merging states are easily put into algorithmic form. Once the minimized state diagram is obtained the inferred grammar can be derived by the following inverse rules from section 6.3.4:

- $N = Q$;
- $S = q_0$;
- $A \mapsto aB$ if $\delta(A, a) = B$ with $A, B \in Q, a \in T$;
- $A \mapsto a$ if $\delta(A, a) = C \in F \subseteq Q$.

6.4 Structural Matching

The key idea in structural matching is to determine the similarity between an unknown pattern and some prototype pattern by using a similarity measure adequate for the structural representations of the patterns.

6.4.1 String Matching

String matching is based on a similarity measure between two strings $\mathbf{x}$ and $\mathbf{y}$ with n and m symbols, respectively, and is defined in terms of the number of elementary

edit operations that are required to transform the string $\mathbf{x}$ into the string $\mathbf{y}$. The most common edit operations are:

1. Substitution of a symbol a in $\mathbf{x}$ by a symbol b in $\mathbf{y}$;
2. Insertion of a symbol a in $\mathbf{y}$;
3. Deletion of a symbol a in $\mathbf{x}$.

We denote these operations as follows:

$$1. a \rightarrow b; \quad (6-18a)$$

$$2. \lambda \rightarrow a; \quad (6-18b)$$

$$3. a \rightarrow \lambda; \quad (6-18c)$$

where λ represents the null string.

Consider, for example, $T = \{h, d, u\}$, $\mathbf{x} = duuh$ and $\mathbf{y} = hdhuh$. String $\mathbf{x}$ can be transformed into $\mathbf{y}$ as follows:

- insert h in $\mathbf{x}$: $hduuh$;
- substitute u in $\mathbf{x}$ by h in $\mathbf{y}$: $hdhuh$.

Another alternative is:

- substitute d in $\mathbf{x}$ by h in $\mathbf{y}$: $huuh$;
- insert d in $\mathbf{x}$: $hduuh$;
- substitute u in $\mathbf{x}$ by h in $\mathbf{y}$: $hdhuh$.

In order to define a matching distance between $\mathbf{x}$ and $\mathbf{y}$, we start by associating costs with the edit operations: $c(e_i)$, where e_i is any one of the edit operations. Next, we compute the total cost of a sequence, s , of k edit operations that transforms $\mathbf{x}$ into $\mathbf{y}$, as:

$$c(s) = \sum_{i=1}^k c(e_i). \quad (6-19)$$

The dissimilarity distance between $\mathbf{x}$ and $\mathbf{y}$ is defined as the minimum cost of all sequences of edit operations that transform $\mathbf{x}$ into $\mathbf{y}$:

$$d(\mathbf{x}, \mathbf{y}) = \min \{c(s) \mid s \text{ transforms } \mathbf{x} \text{ into } \mathbf{y}\}. \quad (6-20)$$

An algorithm for computing this so-called *Levenshtein distance* is based on the enumeration in matrix form of the computation of all sequences of edit operations that transform $\mathbf{x}$ into $\mathbf{y}$.

First, a matrix $D(i, j)$ with $(n+1)$ by $(m+1)$ elements is built, as shown in Figure 6.17. In this matrix, any path from $D(0, 0)$ to $D(n, m)$ represents a sequence of edit operations transforming $\mathbf{x}$ into $\mathbf{y}$. The top row of the matrix is labelled with the $\mathbf{y}$

symbols, and the first column with the symbols of $\mathbf{x}$, with the seed element $D(0, 0) = 0$ representing a situation where no edit operations have been performed. From this seed element, the algorithm proceeds to fill up the matrix by examining the following three predecessors of $D(i, j)$ (see Figure 6.17):

1. $D(i, j-1)$: Progressing from $D(i, j-1)$ to $D(i, j)$ means the insertion of the corresponding top symbol of $\mathbf{y}$ into $\mathbf{x}$, with a cost of $D1 = D(i, j-1) + c(\lambda \rightarrow a)$.
2. $D(i-1, j)$: Progressing from $D(i-1, j)$ to $D(i, j)$ means the deletion of the corresponding left symbol of $\mathbf{x}$, with a cost of $D2 = D(i-1, j) + c(a \rightarrow \lambda)$.
3. $D(i-1, j-1)$: Progressing from $D(i-1, j-1)$ to $D(i, j)$ means the substitution of the corresponding left symbol of $\mathbf{x}$ by the corresponding top symbol of $\mathbf{y}$, with a cost of $D3 = D(i-1, j-1) + c(a \rightarrow b)$. If both symbols are equal, the cost of this operation is zero.

The minimum of $D1$, $D2$ and $D3$ is assigned to $D(i, j)$.

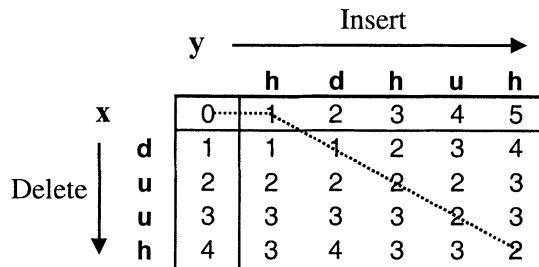


Figure 6.17. Computation of the Levenshtein distance between $\mathbf{x}$ and $\mathbf{y}$, using unitary costs for all elementary edit operations. The dotted line corresponds to the minimum cost path.

Let us use Figure 6.17 to exemplify this procedure, assuming, for simplicity, unitary costs for all edit operations. Applying rules 1 and 2 immediately fills up the top row and left column. Now consider $D(1, 1)$. It can be reached from $D(0, 0)$, $D(0, 1)$ and $D(1, 0)$, adding a cost of one to any of these transitions. If we now consider $D(3, 4)$, we find a minimum cost for the substitution, since both symbols are equal, therefore we obtain $D(3, 4) = D(2, 3)$.

As shown in Figure 6.17 the minimum cost of transforming $\mathbf{x}$ into $\mathbf{y}$ is 2, representing the two previously mentioned edit operations as the first alternative to transforming $\mathbf{x}$ into $\mathbf{y}$. The path corresponding to this sequence of operations is also shown in Figure 6.17.

In order for the Levenshtein distance to have the properties of a metric, the following non-stringent conditions must be satisfied:

1. $c(a \rightarrow b) \geq 0$ for any a, b ;
2. $c(a \rightarrow b) = 0$ if and only if $a = b$;
3. $c(a \rightarrow b) \leq c(\lambda \rightarrow b) + c(a \rightarrow \lambda)$ for any a, b ;
4. $c(a \rightarrow b) = c(b \rightarrow a)$ for any a, b .

In practice, even if these conditions are not satisfied, we still use $d(\mathbf{x}, \mathbf{y})$ as the distance between $\mathbf{x}$ and $\mathbf{y}$. In order to classify patterns described by strings, we can now use the k-nearest neighbour classification approach, described in section 4.3.2, for assigning patterns to classes, given a training set of patterns.

The basic algorithm for computing the Levenshtein distance can be modified in order to cope with a variety of aspects, such as context dependent costs or string attributes (see Bunke, 1990b, for details). Let us consider this last aspect, of matching strings $\mathbf{x}$ and $\mathbf{y}$ with d -dimensional attribute vectors $\mathbf{a}$ and $\mathbf{b}$, respectively. We define a new dissimilarity distance between $\mathbf{x}$ and $\mathbf{y}$ in the following way:

$$d_m(\mathbf{x}, \mathbf{y}) = d_s(\mathbf{x}, \mathbf{y}) + d_a(\mathbf{x}, \mathbf{y}), \quad (6-21)$$

where $d_s(\mathbf{x}, \mathbf{y})$ is the previous dissimilarity distance and $d_a(\mathbf{x}, \mathbf{y})$ is the dissimilarity distance relative to the attributes.

This distance takes account of the attribute differences for the different symbols as:

$$d_a(x, y) = \sum_{r,s} \sum_{i=1}^d w_i |x_{ri} - y_{si}|, \quad (6-21a)$$

where $|x_{ri} - y_{si}|$ takes account of attribute differences for homologous attributes and w_i is a suitably chosen weight.

The summation is taken for all x_r symbols that substitute different y_s symbols. As a matter of fact, this rule can be implemented by inserting an attribute cost into steps 1-3 of the basic algorithm. A particular implementation of this procedure for line segment primitives with length, l , and orientation angle, α attributes, is:

$$c(\lambda \rightarrow a) = I_a + (l_a/m); \quad (6-22a)$$

$$c(a \rightarrow \lambda) = D_a + (l_a/n); \quad (6-22b)$$

$$c(a \rightarrow b) = f(\alpha_a, \alpha_b) + |l_a/n - l_b/m|. \quad (6-22c)$$

where I_a and D_a are the usual insertion and deletion costs. The length attribute is divided by the string length, for the purpose of normalization. Function f takes care of the difference in angular attributes.

Let us see how this applies to the very simple situation shown in Figure 6.18, where the only line segments used are horizontal and vertical. Instead of taking into account an orientation attribute, we use a PDL approach so that the 3 symbols u, h, d with length attribute are used. As the only structural operator used is $+$, we can neglect it when computing distances.

From the results shown in Figure 6.18, obtained with the algorithm implemented in the *String Match.xls* file, some difficulties of this approach are visible. For

instance, the dissimilarity between the prototype and shape (b) is only smaller than the dissimilarity corresponding to shapes (c) or (d) when the substitution cost is sufficiently high compared with the insertion cost. In practical applications, some tuning of elementary costs may therefore be necessary in order to obtain the best results.

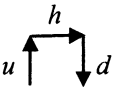
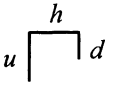
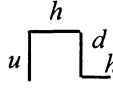
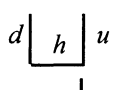
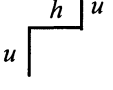
Prototype 		$c_i = 1$	$c_i = 1$	$c_i = 1$	$c_i = \frac{1}{2}$
		$c_d = 1$	$c_d = 1$	$c_d = 1$	$c_d = 1$
		$c_s = \frac{1}{2}$	$c_s = 1$	$c_s = 2$	$c_s = 3$
(a)		0	0	0	0
(b)		1.3	1.3	1.3	1.3
(c)		1	2	4	3.3
(d)		0.5	1	2	2.2

Figure 6.18. Levenshtein distance between a prototype pattern and each of four patterns, (a) to (d). Edit costs are shown at the top.

Despite the difficulties of string matching, this technique can often achieve surprisingly good results in pattern recognition applications. The basic idea consists of obtaining the matching score of an unknown pattern relative to the prototypes, choosing the classification corresponding to the closest prototype. Of course, a generalization of this principle using k-nearest neighbour classification is also possible.

We now illustrate the application of string matching with the recognition of toy models of tanks (*Tanks* dataset). Figure 6.19 shows profile views of three tank prototypes. Figure 6.20 shows one of the prototypes seen at variable distance and orientation, corresponding to the unknown pattern we wish to classify.

String matching is performed using the Levenshtein distance algorithm (*String Match.xls*) with the following modifications: first, because all primitives are attributed line segments, we do not have in this case different string symbols, and a substitution cost is always computed according to (6-21c); second, we now have to deal with an angular attribute measured in $[-180^\circ, 180^\circ]$. We choose a dissimilarity function $f(\alpha_a, \alpha_b)$, which yields a zero cost for angular differences of 0° or 360° and positive increasing cost for other angular differences, up to a maximum of one

for line segments lying in opposite directions. The following function satisfies this requirement:

$$f(\alpha_a, \alpha_b) = 1 - \frac{|\alpha_a - \alpha_b| - 180}{180}. \quad (6-23)$$



Figure 6.19. Three prototype tanks (toy models), labelled T1, T2 and T3, from left to right.



Figure 6.20. Three views of prototype T1 taken at variable distance and perspective angle, labelled T11, T12 and T13, from left to right.

The matching scores between prototypes T1 vs. T2 and T1 vs. T3 are 59 and 57, respectively. Table 6.3 shows the matching scores obtained for each of the "unknown" patterns vs. the prototypes. In all cases, the unknown pattern is correctly classified. It is clear, however, that the discrimination degrades with increasing perspective angle.

Table 6.3. Matching scores between an "unknown" tank and each prototype. The minimum always occurs for the prototype T1.

T11:T1	26
T11:T2	59
T11:T3	56

T12:T1	30
T12:T2	51
T12:T3	48

T13:T1	41
T13:T2	55
T13:T3	48

Improvement of the basic algorithm in order to achieve flexible structural matching, while coping with the issues of scale and orientation changes, as well as contour gaps provoked by occluding objects, is described in the work of Gdalyahu and Weinshall (1999).

The basic ideas of string matching (minimization of a cost function dependent on edit operations) can be generalized and applied to tree or graph matching.

6.4.2 Probabilistic Relaxation Matching

The string matching procedure described in the previous section, performs a sequence of edit operations in order to obtain a match of a string $\mathbf{x}$ with a string $\mathbf{y}$. In some cases, performing the matching sequentially may not be the best idea, since early decisions may completely impair the whole process. In order to see this, let us again consider matching the prototype shape to shapes (b) and (d) in Figure 6.18. The perfect matching of the first two segments of the prototype with shape (d) results in a better matching score than with shape (b) for a not too high substitution cost. The situation may worsen when there are many primitives, and complex relations among them. In such situations, a parallel approach that iteratively adjusts the matching at several points using local constraints may produce better results.

Relaxation labelling is a class of iterative algorithms that works in a parallel way, dealing with several constraints at the same time. In what follows we will describe the two main methods of performing relaxation, *probabilistic* and *discrete*, and we will also present a neural net approach implementing the relaxation.

Let us assume that there are n objects A_i that we want to label with m labels l_k . Suppose also that the label assignments of the objects are interdependent, i.e., the assignments $A_i \rightarrow l_j$ and $A_h \rightarrow l_k$ have some measure of compatibility $c(i;j; h:k)$ that varies in the interval $[-1, 1]$, with 1 meaning full compatibility, -1 full incompatibility and 0 a "don't care" situation. Our goal is to obtain a compatible assignment of labels to objects, measured by a probability estimate, P_{ij} , of assigning l_j to A_i . For this purpose, we have to consider how we can iteratively update our probability estimates in order to obtain a final assignment, starting from an initial situation where the probability of the assignment $A_i \rightarrow l_j$ is $P_{ij}^{(0)}$.

The *probabilistic relaxation* method updates the probabilities of the assignment $A_i \rightarrow l_j$ by noticing first that, given the assignment $A_h \rightarrow l_k$, one would like to increment P_{ij} with $c(i;j; h:k)P_{hk}$ since one wants to increase P_{ij} proportionally to P_{hk} if the compatibility is high and, in the same way, decrease it if the compatibility is low. One way to combine, at each iteration r , the increments for all objects A_h and labels l_k , is to add up the increments for all possible labels as:

$$\sum_{k=1}^m c(i; j; h: k) P_{hk}^{(r)}, \quad (6-24a)$$

and average these increments for all objects $\mathbf{x}_h \neq \mathbf{x}_i$:

$$q_{ij}^{(r)} = \frac{1}{n-1} \sum_{\substack{h=1 \\ h \neq i}}^n \sum_{k=1}^m c(i: j; h: k) P_{hk}^{(r)}. \quad (6-24b)$$

The quantities q_{ij} are in $[-1, 1]$ and can be used to update the P_{ij} , satisfying the constraints that they are nonnegative and add up to one for each object, in the following way:

$$P_{ij}^{(r+1)} = \frac{P_{ij}^{(r)} (1 + q_{ij}^{(r)})}{\sum_{j=1}^m P_{ij}^{(r)} (1 + q_{ij}^{(r)})}. \quad (6-25)$$

The most important decisions one must make when applying probabilistic relaxation relate to the choice of compatibility measures and the estimation of the $P_{ij}^{(0)}$.

Let us consider the so-called image registration problem. This is a pattern recognition problem, where we want to establish a correspondence between two different patterns (images) A and B using two sets of points, called control points. The control points correspond to n points of A and m points of B , respectively, which share k common points, with k unknown. They are chosen so that they reflect clearly identifiable segments of the images. In an application to satellite images registration (Ton and Jain, 1989), the control points are centroids of clearly identifiable regions and have attributes that are the type and the size of the respective region. A matrix of $n \times m$ probabilities P_{ij} can then be established to represent the probability of matching each pair of points. The initial estimates are:

$$P_{ij}^{(0)} = \begin{cases} \min \left(\frac{\text{size}(A_i)}{\text{size}(B_j)}, \frac{\text{size}(B_j)}{\text{size}(A_i)} \right) & \text{if } A_i \text{ and } B_j \text{ are the same type,} \\ 0, & \text{otherwise.} \end{cases} \quad (6-26)$$

The compatibility factors use both the probability estimates and the relative distances among the points:

$$c^{(r)}(i: j; h: k) = P_{hk}^{(r)} \cdot \min \left(\frac{d_{ih}}{d_{jk}}, \frac{d_{jk}}{d_{ih}} \right). \quad (6-27)$$

From these compatibility factors, one can compute the support that the other points lend to the assignment (A_i, B_j) , at iteration r , as:

$$q_{ij}^{(r)} = \frac{1}{m-1} \sum_{\substack{h=1 \\ h \neq i}}^n \left[\max_{\substack{1 \leq k \leq m \\ k \neq j}} c^{(r)}(i: j; h: k) \right]. \quad (6-28)$$

The probabilities are then updated, as:

$$P_{ij}^{(r+1)} = q_{ij}^{(r)} P_{ij}^{(r)}. \quad (6-29)$$

As a result of this updating process, it is expected that the probabilities corresponding to a match will be close to one, while the other values will be close to zero. In practice, one may stop the iterative process when a sufficient discrimination of values above or below 0.5 is reached.

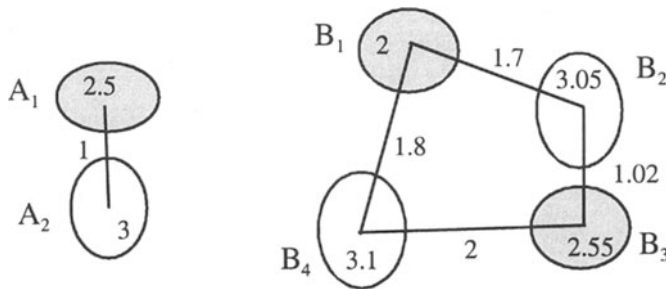


Figure 6.21. An image registration example, where the goal is to establish a correspondence of centroids of A regions with centroids of B regions.

We exemplify this method using the regions shown in Figure 6.21, where there are two types of regions (white and grey) with areas and distances among the centroids as indicated. Table 6.4 shows the probability matrix from the initial phase up to the second iteration. Initially, the probabilities depend only on the region size, and in this particular case, all non-zero values are initially close to one. In the following iterations, the compatibility factors play their role so that in the second iteration it is already clear that there is only one valid match: (A_1, B_3) , (A_2, B_2) .

Table 6.4. Estimates of centroid matching probabilities for the image registration example.

	$P^{(0)}$			$P^{(1)}$			$P^{(2)}$	
	A_1	A_2		A_1	A_2		A_1	A_2
B_1	0.8	0	B_1	0.463	0	B_1	0.257	0
B_2	0	0.984	B_2	0	0.945	B_2	0	0.876
B_3	0.980	0	B_3	0.945	0	B_3	0.876	0
B_4	0	0.968	B_4	0	0.474	B_4	0	0.224

In this case, the updating of the probabilities is quite easily done by hand. For instance, the first updating of P_{11} depends only on the compatibilities of (A_2, B_2) and of (A_2, B_4) , with:

$$\begin{aligned} c(1:1; 2:2) &= P_{22} (1/1.7) = 0.5788; \\ c(1:1; 2:4) &= P_{24} (1/1.8) = 0.5378. \end{aligned}$$

The maximum value of these compatibilities for A_2 , updates $P_{11} = 0.8$ to $P_{11} = 0.8 \times 0.5788 = 0.463$.

This basic probabilistic relaxation scheme can also be applied to other pattern recognition tasks, including object classification, as exemplified in Exercise 6.22.

6.4.3 Discrete Relaxation Matching

In some situations it is not adequate to assign probabilities to the object labelling, since the only decision is whether the label is possible or not, i.e., P_{ij} is either 1 or 0. Usually, in these situations the compatibilities are also qualitative in nature: the $A_i \rightarrow I_j$ assignment is either compatible ($c=1$) or not ($c=0$). An example of such a situation is in graph matching, where correspondence between nodes is either possible or impossible.

There is a variant of the basic relaxation algorithm, called *discrete relaxation*, appropriate for these kinds of situations. In discrete relaxation, for every A_h relevant to A_i , there must be at least one label k , such that $c(i:j; h:k)=1$ with $P_{hk}=1$; otherwise the assignment $A_i \rightarrow I_j$ is impossible. Initially, the $P_{ij}^{(0)}$ are all set to one. At subsequent iterations, P_{ij} remains 1 if for every relevant A_h there is an assignment such that $c(i:j; h:k)P_{hk}=1$; otherwise P_{ij} becomes 0. This is equivalent to the following formula:

$$P_{ij}^{(r+1)} = \min_h \left[\max_{k=1}^m c(i:j; h:k) P_{hk}^{(r)} \right]. \quad (6-30)$$

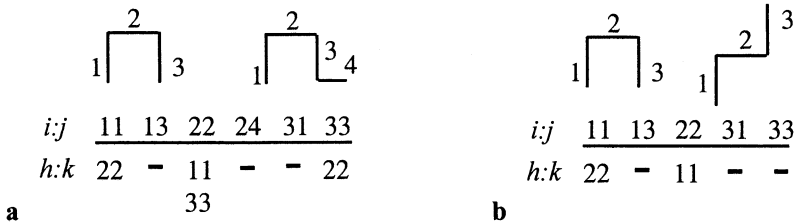


Figure 6.22. Discrete relaxation applied to two simple examples of compatible (a) and incompatible (b) shapes.

Formula (6-30) estimates possible assignments for A_i by checking that each assignment is compatible with the surviving assignments of A_i 's neighbours.

Let us apply discrete relaxation to the matching of the prototype shape of Figure 6.18 with shapes (b) and (d).

In this example we only distinguish whether the lines are horizontal or vertical, and consider that an assignment $i:j$ is compatible if they are of the same type. Further, $i:j$ is compatible with $h:k$ if h is an immediate neighbour of i , such that it is possible to find a translation $i:j$ that superimposes h over k . With these compatibility rules, we can list the compatible assignments as shown in Figure 6.22 for each pair $i:j$. For instance, for the matching (a), the assignment 1:1 is compatible with 2:2; for the assignment 1:3, one does not find any compatible primitives, since the neighbour segment 2 of the first shape is not compatible with the neighbour segment 4 of the second shape.

Starting with a matrix $[P_{ij}]$ filled in with ones, we can now apply the compatibility rules to arrive at the final matrix shown in Table 6.5 for the matching situation of Figure 6.22a. It is clear that a total match is reached (1:1, 2:2, 3:3). For the shape matching situation of Figure 6.22b, only the partial match (1:1, 2:2) is obtained.

Notice that for complex shapes, the computation of the compatibilities can be cumbersome. We will see in the next section a method to expedite their computation.

Table 6.5. Probability matrix for the matching situation of Figure 6.22a.

$\mathbf{P}^{(0)}$

	1	2	3	4
1	1	1	1	1
2	1	1	1	1
3	1	1	1	1

$\mathbf{P}^{(1)}$

	1	2	3	4
1	1	0	1	0
2	0	1	0	1
3	1	0	1	0

$\mathbf{P}^{(2)}$

	1	2	3	4
1	1	0	0	0
2	0	1	0	0
3	0	0	1	0

6.4.4 Relaxation Using Hopfield Networks

Relaxation methods can be implemented using neural networks. In particular, a modified version of the Hopfield network has been found useful in several applications of relaxation methods, namely in line drawing matching (Kuner and Ueberreiter, 1988) and in hierarchical recognition of three-dimensional objects such as machine parts (Lin *et al.*, 1991).

We will now describe how this approach is applied using the line drawings example. Imagine that we have two line drawings X and Y, constituted by line segments x_i ($i = 1, \dots, n$) and y_j ($j = 1, \dots, m$). The line drawing X is considered the prototype drawing that we want to match with a subset of the line drawing of Y.

For this purpose, we start by defining a function $t(x_i, x_h)$ that reflects the transformation that must be applied to x_i in order to obtain x_h . For some applications, function $t(x_i, x_h)$ just computes a matching score resembling the transforming function used in (6-22c).

Next, we compute the compatibility coefficients for the assignments $x_i \rightarrow y_j$ and $x_h \rightarrow y_k$ as follows:

$$c(i: j; h: k) = 1 \quad \text{if} \quad t(x_i, x_h) = t(y_j, y_k); \quad (6-31a)$$

$$c(i: j; h: k) < 1 \quad \text{if} \quad t(x_i, x_h) \neq t(y_j, y_k); \quad (6-31b)$$

$$c(h: k, i: j) = c(i: j; h: k). \quad (6-31c)$$

Therefore, full compatibility only exists if the matching between candidate segments in the prototype drawing is the same as between candidate segments in the unknown drawing.

Let us use a Boolean matrix $\mathbf{M}$ with n lines and m columns for the representation of the possible assignments of line segments of X to line segments of Y . Possible assignments are represented by $m_{ij}=1$, impossible assignments by $m_{ij}=0$. A matching of X with a subset of Y corresponds to a $n \times m$ matrix $\mathbf{X}$, generated from $\mathbf{M}$, such that:

$$x_{ij} = 1 \Rightarrow x_{il} = 0 \quad \text{if} \quad l \neq j, \quad (6-32)$$

since in any matching, a segment of X can only be assigned to one segment of Y .

The optimal assignment corresponds to:

$$\max \left(\sum_{i=1}^n \sum_{h=1}^n \sum_{j=1}^m \sum_{k=1}^m x_{ij} c(i: j; h: k) x_{hk} \right), \quad (6-33a)$$

$$\text{with } \sum_{j=1}^m x_{ij} = 1 \quad \text{and} \quad \sum_{i=1}^n x_{ij} \leq 1. \quad (6-33b)$$

This is a quadratic assignment problem that can be solved using a probabilistic relaxation method (see Kuner and Ueberreiter, 1988). Initially, a probability matrix is initialised with a constant value, e.g. $1/n$. Next, the basic relaxation algorithm expressed by formulas (6-24) and (6-25) is applied until there is no significant difference between the probability matrices $\mathbf{P}$ in successive iterations. Matrix $\mathbf{P}$ represents then the probabilities of the assignments, with the higher values P_{ij} representing the best assignments $x_i \rightarrow y_j$. One then proceeds to solve the much simpler linear assignment problem:

$$\max \left(\sum_{i=1}^n \sum_{j=1}^m P_{ij} x_{ij} \right) \quad \text{with the constraints (6-33b)}. \quad (6-34)$$

This relaxation assignment can be solved with a modified Hopfield net in the following way:

1. Set a two-dimensional Hopfield network with the rows representing the segments (objects) of X and the columns the segments (objects) of Y. Each node of the net, now denoted x_{ij} , represents an assignment $x_i \rightarrow y_j$;
2. Compute the connection weights of the net as $w_{ijhk} = c(i:j, h:k)$, which correspond to the weight that links neuron x_{ij} with x_{hk} ;
3. Initialise the inputs with a matrix $\mathbf{M}$ of possible assignments, $m_{ij} = 1/n$, and set a total compatibility factor $q=0$;
4. Perform the output updating of the network in a full parallel way:

$$P_{ij} = \text{sgn} \left(\sum_{h=1}^n \sum_{k=1}^m c(i:j, h:k) m_{hk} \right); \quad (6-35)$$

5. With the probability matrix obtained, solve the linear assignment problem (6-34) and evaluate the new value of q , q' , using the objective function of (6-33a).
6. If $q' < q$ set $\mathbf{P} = \mathbf{M}$ and stop, otherwise set $\mathbf{M} = \mathbf{P}$, $q = q'$ and go to step 4.

The *Hopfield* program (Appendix B) allows one to perform experiments of a Hopfield net in the particular case of discrete relaxation. Let us consider the simple example shown in Figure 6.22a. In order to derive the optimal assignment, we start by setting a network of 3×4 neurons. Next, since we are using the values $\{1, -1\}$, instead of $\{1, 0\}$ for the discrete Hopfield net, we set all the weights to -1 (corresponding to incompatibility) except for the compatible assignments, where we set the corresponding weight to the value $n \times m = 12$, therefore insuring a positive result if at least one of the products $c(i:j, h:k) m_{hk}$ is positive. We initialise the net with a grid filled in with ones and get the final solution shown in Table 6.5, where the final assignment is easily derived.

Let us now consider a less trivial example of having to find the compatible assignment between the line drawings shown in Figure 6.23. We start by defining the function that reflects the transformation that must be applied to segment x_i in order to obtain segment x_h . For this purpose, we may choose the following vectorial function:

$$\mathbf{t}(x_i, x_k) = [d_{\min} \quad r_l \quad \alpha]^t, \quad \text{with} \quad (6-36)$$

$d_{\min}$: minimum distance between segments;
 r_l : ratio of the segments lengths;
 α : angle between segments, in $[0, 180^\circ]$.

With this function it is easy to see, for instance, that the pair $(\mathbf{x}_2, \mathbf{x}_3)$ is compatible with $(\mathbf{y}_4, \mathbf{y}_6)$ but not with $(\mathbf{y}_5, \mathbf{y}_6)$. When applying the comparisons

(6-31a) and (6-31b), it is convenient to set some tolerance for all components of $\mathbf{t}(x_i, x_k)$.

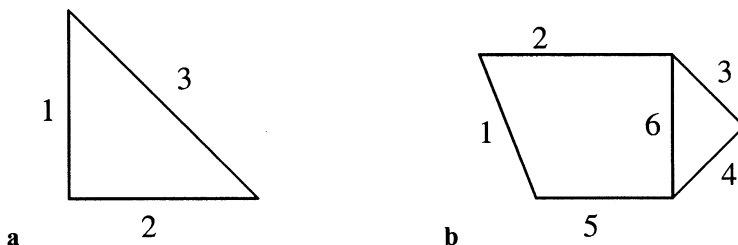


Figure 6.23. Line drawings for illustration of discrete matching with a Hopfield network: (a) prototype; (b) "unknown" drawing.

The compatibility matrix corresponding to the Hopfield net weights is a sparse matrix, since it is filled with -1 (incompatible assignment) for all elements w_{ijhk} such that $h=i$ or $k=j$ (shaded cells in Table 6.6). Also, since it is a symmetric matrix, one only has to fill in half of the matrix values. Table 6.6 shows part of the compatibility matrix using the binary set $\{1, 0\}$. The Hopfield weight matrix will be filled with -1 instead of 0, and $n \times m = 18$ instead of 1. For instance, since $(\mathbf{x}_2, \mathbf{x}_3)$ is compatible with $(\mathbf{y}_4, \mathbf{y}_6)$, the weight w_{2436} will be filled in with 18.

Table 6.6. Compatibility matrix for the example of Figure 6.23. Parts of the matrix not shown are filled in with zeros.

	21	22	23	24	25	26	31	32	33	34	35	36
11	0	0	0	0	0	0	0	0	0	0	0	0
12	0	0	0	0	0	0	0	0	0	0	0	0
13	0	0	0	1	0	0	0	0	0	0	0	1
14	0	0	1	0	0	0	0	0	0	0	0	1
15	0	0	0	0	0	1	0	0	0	0	0	0
16	0	0	0	0	1	0	0	0	0	0	0	0

	21	22	23	24	25	26	31	32	33	34	35	36
21	0	0	0	0	0	0	0	0	0	0	0	0
22		0	0	0	0	0	0	0	0	0	0	0
23			0	0	0	0	0	0	0	0	0	1
24				0	0	0	0	0	0	0	0	1
25					0	0	0	0	0	0	0	0
26						0	0	0	0	0	0	0

With the Hopfield network inputs set initially with ones, the net will iterate to the outputs shown in Figure 6.24a, which give a picture of all feasible assignments. We now solve the linear assignment problem (6-34), also using the Hopfield net. Since $\mathbf{x}_3 \rightarrow \mathbf{y}_6$ is the only feasible assignment for $\mathbf{x}_3$, we investigate other assignments satisfying (6-33b). One such assignment is shown in Figure 6.23b and

has a final state shown in Figure 6.23c, with $q=2$, which corresponds to the partial assignment $\{x_3 \rightarrow y_6, x_2 \rightarrow y_5\}$. The maximum value of $q=3$ is only obtained with the full compatible assignments $\{x_3 \rightarrow y_6, x_2 \rightarrow y_3, x_1 \rightarrow y_4\}$ or $\{x_3 \rightarrow y_6, x_2 \rightarrow y_4, x_1 \rightarrow y_3\}$.



Figure 6.24. Hopfield net illustrating the probabilities associated with the assignments corresponding to the example shown in Figure 6.23. (a) Feasible assignments as solutions to the quadratic problem; (b) A candidate assignment; (c) final assignment corresponding to the previous situation ($q=2$).

The basic ideas that have just been described, concerning the application of a discrete Hopfield network for discrete relaxation matching, can be generalized for more complex probabilistic relaxation tasks, using a modified version of the continuous Hopfield network. In the work of Lin *et al.* (1991), the hierarchical recognition of machine parts using such modified Hopfield networks is described. The hierarchical process uses: at the first stage, a Hopfield network whose weights correspond to surface matching scores, picking up possible candidates from a database of machine parts; at the second stage, a Hopfield network performs vertex matching in order to achieve a finer selection.

6.4.5 Graph and Tree Matching

Consider two graphs (or trees) $G_1=\{N_1, R_1\}$ and $G_2=\{N_2, R_2\}$, describing the structural relations of two patterns. A *homomorphism* from G_1 to G_2 is any function f establishing a correspondence between the nodes of N_1 and the nodes of N_2 such that any pair of adjacent nodes in G_1 corresponds, through f , to adjacent nodes in N_2 . Let (a, b) represent two adjacent nodes of N_1 . Then a homomorphism f satisfies:

$$(a, b) \in R_1 \Rightarrow (f(a), f(b)) \in R_2. \quad (6-37)$$

Consider the two digraphs represented in Figure 6.25. When assessing node adjacency, we have to take into consideration the direction of the arc connecting the nodes. There are two easily found homomorphisms between both digraphs.

Two similar patterns will have a homomorphic relation in at least a large part of their graph or tree representations. The two patterns will be structurally equivalent if an *isomorphism* exists:

$$(a, b) \in R_1 \Leftrightarrow (f(a), f(b)) \in R_2. \quad (6-38)$$

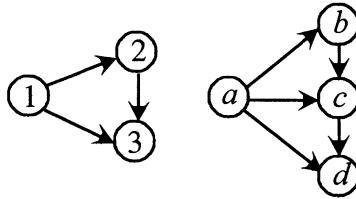


Figure 6.25. Two digraphs with two homomorphisms: (a) $f(1)=a, f(2)=b, f(3)=c$; (b) $f(1)=a, f(2)=c, f(3)=d$.

The structural descriptions are then the same except for a node re-labelling operation.

An algorithm for finding homomorphisms in two graphs $G_1=\{N_1, R_1\}$ and $G_2=\{N_2, R_2\}$ is based on the idea of first building the so-called *match graph*. This is a graph where each node represents an assignment from one of the N_1 nodes to one of the N_2 nodes. Let (n_1, n_2) and (n_1', n_2') be two such assignments, representing two distinct nodes in the match graph. An arc exists between these two nodes if the assignments are compatible or, equivalently, if the relation for the node pair (n_1, n_1') is the same as for the node pair (n_2, n_2') .

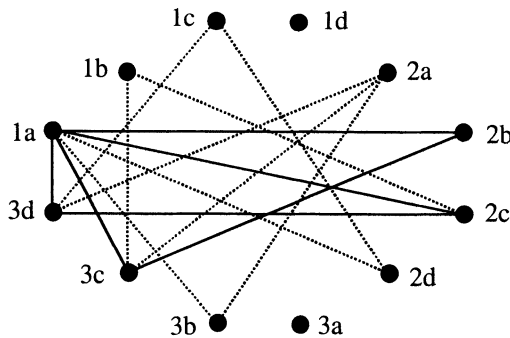


Figure 6.26. Match graph for the two graphs shown in Figure 6.25. The homomorphisms are shown with solid line.

Once the match graph is built, as shown in Figure 6.26 for the two graphs of Figure 6.25, one proceeds to determine its *cliques*. A clique is a totally connected sub-graph between any two different nodes, where totally connected means that there is a path linking any given pair of nodes. In our case, we are interested in maximal cliques involving different nodes of the source graphs. These are shown with solid line in Figure 6.26, representing the two known homomorphisms.

As seen in this example, finding isomorphisms or homomorphisms between graphs or trees can be a hard task in non-trivial situations, since it demands searching for compatible node adjacencies in the space of all possible node correspondences. This usually amounts to a long time-consuming task, for anything except a very small number of nodes. Existing algorithms that perform this search require a time that is approximately factorial-bound with the maximum number of nodes for the target correspondence. No polynomial-bound algorithms are known. The situation worsens when one has to assess not only node adjacency, but also attribute compatibility. For instance, if instead of the digraphs of Figure 6.25 we consider the digraphs of Figure 6.4, no homomorphism exists. However, an evident homomorphism exists between the digraphs of "F" and "E".

Practical applications of pattern recognition, with patterns described by trees or graphs, resort to faster methods of assessing the structural dissimilarity, namely to the idea of edit cost as explained in section 6.4.1 or to discrete relaxation as explained in section 6.4.3.

Tree matching based on edit operations is conceptually similar to the string matching method described in section 6.4.1. The following five edit operations are used:

1. Insertion of a node
2. Deletion of a node
3. Substitution of a node label
4. Splitting a node
5. Merging two nodes

Algorithms that assess tree dissimilarity using these edit operations are described in Sanfeliu (1990). These algorithms have a time complexity of the order of nm^2 , where n and m are the number of nodes of the trees to be matched.

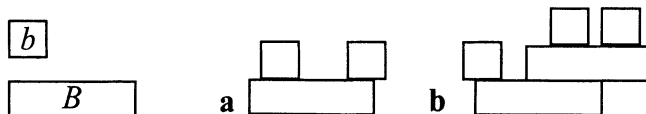


Figure 6.27. Two scene situations in a "block world", with primitives small block, b , and large block, B .

Relaxation techniques are quite attractive for assessing tree or graph dissimilarity, since they can be executed quickly and easily incorporate any constraints, namely those derived from the attributes. In some situations of tree or graph matching it is sufficient to perform discrete relaxation, much in the same way as we did in the previous sections. Consider for instance the scene situations shown in Figure 6.27. These can be represented by the trees of Figure 6.28, where a descendent node means "above". We can now perform discrete relaxation using the compatibilities of the neighbour nodes also shown in Figure 6.28. The probability matrix of the assignments evolves as shown in Table 6.7. A homomorphism of scene (a) relative to scene (b) is detected.

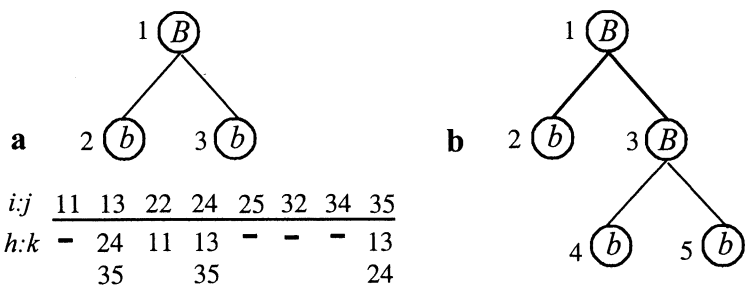


Figure 6.28. Trees describing the scene situations of Figure 6.27, with the compatibility table.

Table 6.7. Probability matrix of the tree matching problem shown in Figure 6.28.

$P^{(0)}$						$P^{(1)}$						$P^{(2)}$					
	1	2	3	4	5		1	2	3	4	5		1	2	3	4	5
1	1	1	1	1	1	1	1	0	1	0	0	1	0	0	1	0	0
2	1	1	1	1	1	0	1	0	1	1		0	0	0	1	0	
3	1	1	1	1	1	0	1	0	1	1		0	0	0	0	0	1

The Hopfield network approach can also be used for determining the graph homomorphisms, in the same way as explained in the previous section (see Exercise 6.24).

For attributed relational graphs such as those exemplified in Figure 6.4, one would have to compute similarities for the node properties (v, h, u, d, c, b) and for the relational properties (a, e, l, t). These would influence, through appropriate weighting, the compatibilities among the graph nodes. Work along these lines for

generalized attributed relational graphs has been developed by Li (1992). In his work, a relaxation matching algorithm invariant to translations, rotations and scale changes is presented.

Special types of recurrent neural networks have also been developed for acyclic graph matching. Their application to automatic inference in syntactic and structural pattern recognition is described by Sperduti *et al.* (1996).

Bibliography

- Biland HP, Wahl FM (1988) Hough-Space Decomposition for Polyhedral Scene Analysis. In: Ferraté G, Pavlidis T, Sanfeliu A, Bunke H (eds) Syntactic and Structural Pattern Recognition. Springer-Verlag.
- Bunke H (1990a) String Grammars for Syntactic Pattern Recognition In: Bunke H, Sanfeliu A (eds) Syntactic and Structural Pattern Recognition. Theory and Applications. World Scientific Pub. Co., Singapore.
- Bunke H (1990b) String Matching for Structural Pattern Recognition. In: Bunke H, Sanfeliu A (eds) Syntactic and Structural Pattern Recognition. Theory and Applications. World Scientific Pub. Co., Singapore.
- Bunke H (1990c) Hybrid Pattern Recognition Methods. In: Bunke H, Sanfeliu A (eds) Syntactic and Structural Pattern Recognition. Theory and Applications. World Scientific Pub. Co., Singapore.
- Chen JW, Lee SY (1996) A Hierarchical Representation for the Reference Database of On-line Chinese Character Recognition. In: Perner P, Wang P, Rosenfeld A (eds) Advances in Structural and Syntactical Pattern Recognition, Springer Verlag, pp. 351-360.
- Davis LS (1979) Shape Matching Using Relaxation Techniques. IEEE Tr Patt An Mach Intell, 1:60-72.
- DeMori R (1977) Syntactic Recognition of Speech Signals. In: Fu KS (ed) Syntactic Pattern Recognition Applications. Springer-Verlag.
- Di Baja GS (1996) Representing Shape by Line Patterns. In: Perner P, Wang P, Rosenfeld A (eds) Advances in Structural and Syntactical Pattern Recognition, Springer Verlag, pp. 230-239.
- Duda RO, Hart PE (1973) Pattern Classification and Scene Analysis. Wiley, New York.
- Fu KS (1977) Introduction to Syntactic Pattern Recognition. In: Fu KS (ed) Syntactic Pattern Recognition Applications. Springer-Verlag.
- Gdalyahu Y, Weinshall D (1999) Flexible Syntactic Matching of Curves and its Application to Automatic Hierarchical Classification of Silhouettes. IEEE Tr Patt An Mach Intell, 21:1312-1328.
- Gold S, Rangarajan A (1996) A Graduated Assignment Algorithm for Graph Matching. IEEE Tr Patt An Mach Intell, 18:377-388.
- Horowitz SL (1977) Peak Recognition in Waveforms. In: Fu KS (ed) Syntactic Pattern Recognition Applications. Springer-Verlag.
- Hummel RA, Zucker SW (1983) On the Foundations of Relaxation Labeling Processes. IEEE Tr Patt An Mach Intell, 5:267-285.
- Kuner P, Ueberreiter B (1988) Pattern Recognition by Graph Matching. Combinatorial Versus Continuous Optimization. Int. J. Patt. Rec. Artif. Intell., 2: 527-542.

- Lapresté JT, Cartoux JY, Richetin M (1988) Face Recognition from Range Data by Structural Analysis. In: Ferraté G, Pavlidis T, Sanfeliu A, Bunke H (eds) Syntactic and Structural Pattern Recognition. Springer-Verlag.
- Lee KH, Eom KB, Kashyap RL (1992) Character Recognition based on Attribute-Dependent Programmed Grammar. IEEE Tr. Patt. An. Mach. Intell., 14:1122-1128.
- Li SZ (1992) Matching: Invariant to Translations, Rotations and Scale Changes. Pattern Recognition, 25:583-594.
- Lin WC, Liao FY, Tsao CK, Lingutla T (1991) A Hierarchical Multiple-View Approach to Three-Dimensional Object Recognition. IEEE Tr Neural Networks, 2:84-92.
- Mehrotra R, Grosky WI (1988) Smith: An Efficient Model-Based Two Dimensional Shape Matching Technique. In: Ferraté G, Pavlidis T, Sanfeliu A, Bunke H (eds) Syntactic and Structural Pattern Recognition. Springer-Verlag, pp. 233-248.
- Miclet L (1990) Grammatical Inference. In: Bunke H, Sanfeliu A (eds) Syntactic and Structural Pattern Recognition. Theory and Applications. World Scientific Pub. Co., Singapore.
- Miclet L, Quinqueton J (1988) Learning from Examples in Sequences and Grammatical Inference. In: Ferraté G, Pavlidis T, Sanfeliu A, Bunke H (eds) Syntactic and Structural Pattern Recognition. Springer-Verlag.
- Moayer B, Fu KS (1977) Fingerprint Classification. In: Fu KS (ed) Syntactic Pattern Recognition Applications. Springer-Verlag.
- Nishida H (1996) A Structural Analysis of Curve Deformation by Discontinuous Transformations. In: Perner P, Wang P, Rosenfeld A (eds) Advances in Structural and Syntactical Pattern Recognition, Springer Verlag, pp. 269-278.
- Pao DCW, Li HF (1992) Shapes Recognition Using the Straight Line Hough Transform: Theory and Generalization. IEEE Tr Patt An Mach Intell, 14:1076-1128.
- Pavlidis T (1973) Waveform Segmentation Through Functional Approximation. IEEE Tr Computers, 22:689-697.
- Rosenfeld A, Kak A (1982) Digital Picture Processing, vol. 2. Academic Press Inc., Orlando, Florida.
- Rosenfeld A (1990) Array, Tree and Graph Grammars. In: Bunke H and Sanfeliu A (eds) Syntactic and Structural Pattern Recognition. Theory and Applications. World Scientific Pub. Co., Singapore.
- Sanfeliu A (1990) Matching Tree Structures. In: Bunke H, Sanfeliu A (eds) Syntactic and Structural Pattern Recognition. Theory and Applications. World Scientific Pub. Co., Singapore.
- Schalkoff R (1992) Pattern Recognition. Statistical, Structural and Neural Approaches. John Wiley & Sons, Inc.
- Shapiro L (1988) Ordered Structural Matching. In: Ferraté G, Pavlidis T, Sanfeliu A, Bunke H (eds) Syntactic and Structural Pattern Recognition. Springer-Verlag.
- Shapiro LG, Haralick RM (1990) Matching Relational Structures Using Discrete Relaxation. In: Bunke H, Sanfeliu A (eds) Syntactic and Structural Pattern Recognition. Theory and Applications. World Scientific Pub. Co., Singapore.
- Skordalakis E (1990) ECG Analysis. In: Bunke H, Sanfeliu A (eds) Syntactic and Structural Pattern Recognition. Theory and Applications. World Scientific Pub. Co., Singapore.
- Sperduti A, Majidi D, Starita A (1996) Extended Cascade-Correlation for Syntactic and Structural Pattern Recognition. In: Perner P, Wang P, Rosenfeld A (eds) Advances in Structural and Syntactical Pattern Recognition, Springer Verlag, pp. 90-99.

- Stenstrom JR (1988) Training and Model Generation for a Syntactic Curve Network Parser. In: Ferraté G, Pavlidis T, Sanfeliu A, Bunke H (eds) Syntactic and Structural Pattern Recognition. Springer-Verlag, pp. 249-267.
- Tai JW, Liu YJ (1990) Chinese Character Recognition. In: Bunke H, Sanfeliu A (eds) Syntactic and Structural Pattern Recognition. Theory and Applications. World Scientific Pub. Co., Singapore.
- Thomason GT (1990) Introduction and Overview. In: Bunke H, Sanfeliu A (eds) Syntactic and Structural Pattern Recognition. Theory and Applications. World Scientific Pub. Co., Singapore.
- Ton J, Jain AK (1989) Registering Landsat Images by Point Matching. IEEE Tr Geosci and Rem Sensing, 27:642-651.
- Tsai WH (1990) Combining Statistical and Structural Methods. In: Bunke H, Sanfeliu A (eds) Syntactic and Structural Pattern Recognition. Theory and Applications. World Scientific Pub. Co., Singapore.
- Vámos T (1977) Industrial Objects and Machine Parts Recognition. In: Fu KS (ed) Syntactic Pattern Recognition Applications. Springer-Verlag.
- Wong AKC, Constant J, You ML (1990) Random Graphs. In: Bunke H and Sanfeliu A (eds) Syntactic and Structural Pattern Recognition. Theory and Applications. World Scientific Pub. Co., Singapore.
- You KC, Fu KS (1979) A Syntactic Approach to Shape Recognition Using Attributed Grammars. IEEE Tr Systems, Man and Cybernetics, 9:334-345.

Exercises

- 6.1 Consider the grammar presented in (6-7d).
- What type of grammar is it?
 - Two grammars G_1 , G_2 are considered equivalent if $L(G_1) = L(G_2)$. Is there a regular grammar equivalent to (6-7d)? Why?
- 6.2 A grammar for waveform description has the following production rules:
- $$\begin{aligned} uP^+ &\mapsto u(h \mid u)(P^+ \mid \lambda)d ; \\ hP^+ &\mapsto hu(P^+ \mid \lambda)d . \end{aligned}$$
- Interpret the production rules. What type of grammar is it?
 - Which of the following sequences is parsed by the grammar: $hhud$; $huuhdd$?
- 6.3 Modify the production rules (6-12a) so that the corresponding grammar will recognize positive and negative spikes formed by an arbitrary number of D and U primitives, with at most an h primitive between.
- 6.4 Consider the symbol set $T = \{h, p\}$ used to describe signals, where h represents a horizontal segment and p is a positive peak. Build a regular grammar capable of recognizing sequences terminated by hph , i.e., by isolated peaks. Draw the state diagram of the finite-state automaton that recognizes such sequences.

- 6.5 Improve the set of primitives and relations indicated in section 6.2.2 (Figure 6.4) in order to describe all capital letters by digraphs.
- 6.6 In section 6.3.2 two different PDL descriptions were given for the pattern "R", according to the traversing direction of the description. Develop a general rule that will generate a PDL description based on the description of the inverse traversing direction.
- 6.7 Consider "standard" prototype drawings of the decimal digits.
- Develop a PDL grammar that will generate the prototypes for each digit class.
 - Perform string matching experiments (use *StrMatch.xls*) in order to determine which digit class an unknown PDL sequence can be assigned to. Also set a threshold for the matching score in order to establish a reject class.
- 6.8 An FHR spike (see Figure 6.11) has amplitude that is approximately half (negative spike) or double (positive spike) of its onset value.
- Determine the attributed grammar that will detect FHR spikes.
 - Perform experiments with this grammar using the *FHR Signals* dataset, adjusting the appropriate attribute rules.
- 6.9 An upward FHR wave is considered an acceleration if it lasts for more than 15 s, reaching an amplitude of at least 15 bpm. Write the appropriate rules for the attributed grammar describing FHR accelerations (see rules (6-14)).
- 6.10 A downward FHR wave is considered a deceleration if it lasts for more than 15 s, reaching an amplitude of at least 10 bpm.
- Draw the state diagram describing decelerations, analogous to the one describing accelerations shown in Figure 6.12.
 - Write the appropriate rules for the attribute vectors of this grammar, studying its behaviour by experimenting with the *FHR Signals* dataset.
- 6.11 Develop the grammar rules that will recognize the P, Q, R, S and T waves of ECG signals (see description in A.4). Use the *ECG Signals* dataset to carry out experiments with these rules.
- 6.12 A human expert has assigned the waves described by the following strings of $T=\{h, u, d\}$ to a certain class:

hudh
huudh
huhuhddh
huhudh
huhuuddh

Use the successor method in order to determine the state diagram that will recognize these strings.

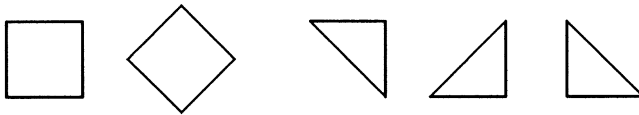
- 6.13 Repeat the tank classification experiments, described in section 6.4.1, using other views of tanks available in the *Tanks* dataset. Determine whether an improvement of matching scores can be obtained by using scale and angular normalizations derived

from alignment of segment references. Use as segment reference the upper edge of the tank cannon.

- 6.14 Use an attributed string matching approach for the detection of P waves in ECG signals, selecting one of the waves as prototype. Perform experiments using the column G signal of *ECG Signals.xls*, characterised by the presence of multiple P waves.
- 6.15 An impulsive signal sometimes appears buried in noise. The signal impulses can be described by:

$$\begin{array}{ccccccc} 0.3 & 0.7 & 0.3 & 0.7 \\ S \mapsto hS; & S \mapsto uP; & P \mapsto dF; & P \mapsto uP; \\ 0.2 & 0.4 & 0.4 \\ F \mapsto dF; & F \mapsto hS; & F \mapsto uP. \end{array}$$

- Draw the state diagram corresponding to these rules.
 - What is the probability of the sequence *uudud* appearing, and what are the sequences most favoured by the transition probabilities?
 - The presence of noise enhances the occurrence rate of saw-tooth sequences of the *udud* type. Modify the transition probabilities so that such sequences have always a probability of occurring below 0.05.
- 6.16 Consider the chain code description of object contours, presented in section 6.1.2.



- Write the string descriptions of the shapes above, assuming that the description starts with the upper left point.
 - Assuming that all sides are equal, as for the square, what is the correspondence rule between descriptions of the same object rotated by multiples of 45° ?
 - Taking into consideration the descriptions of the triangles, how should one modify the previous rule in order to obtain the same chain code descriptions of any object rotated by multiples of 45° ?
 - How must one modify the algorithm that computes the Levenshtein distance of such chain code strings, so that the matching score obtained is invariant to object rotations by multiples of 45° ?
- 6.17 Consider the tree description of cupboards, as exemplified in Figure 6.6. Discard the body primitive and the *o*, *i* relations. Assume that the cupboards are built in a modular way, with modules constituted by at most two pieces (e.g., a module can have two shelves, another can have a drawer either on the top or on the bottom, etc.). Furthermore, there are equivalent modules; for instance, a module with a top shelf is considered equivalent to another one with a bottom shelf. Develop a discrete relaxation

scheme that will be able to determine whether a cupboard includes a given assembly of modules or not.

6.18 Consider the tree description of cupboards as in the previous Exercise.

a) Show that the cupboards can also be described by a string grammar using the symbols of the primitives, plus:

Symbol operators: $\leftarrow$ (left of) ; $\uparrow$ (top of)

Parentheses indicating the precedence order of the operations: ()

b) Interpret: $(s \uparrow d) \leftarrow (l \uparrow l)$; $((s \uparrow s) \leftarrow e) \uparrow ((d \uparrow d) \leftarrow (l \uparrow r))$.

c) The string grammar description of cupboards can discard the parentheses by using a *reverse Polish notation*, where the string is scanned from right to left, and whenever an operator is encountered, the corresponding operation is performed upon the previous operands. An example of this notation is $\leftarrow \uparrow s d \uparrow l l$ for the first string of b). Using this notation, what string corresponds to the second string of b)?

d) Perform string matching experiments using strings in reverse Polish notation to describe cupboards.

6.19 Develop a probabilistic relaxation scheme for the classification of profile silhouettes of tanks, using the line segment description presented in section 6.4.1 and available in the *Tanks* dataset.

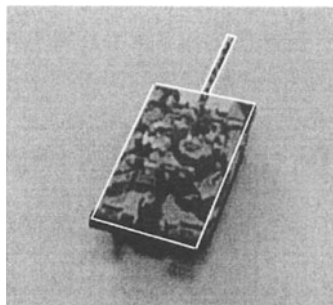
6.20 A common operation performed on ECG signals consists of temporal alignment of the respective QRS wave sequences (see description in A.4) in order to obtain averaged waves. Consider the string descriptions of QRS wave sequences, using h , u , d , U and D primitives, obtained by the *SigParse* program for the ECGs of the *ECG Signals* dataset. Perform temporal alignment experiments of QRS waves using:

a) String matching.

b) Probabilistic relaxation.

6.21 Recompute the probability matrix $\mathbf{P}$ of Table 6.4, assuming that the distance between the centroids of A_1 , A_2 (Figure 6.21) is 2.2.

6.22 Consider the aerial images of tanks, included in the *Tanks* dataset, with two prototype images TU4 and TU5.



Use probabilistic relaxation in order to classify an "unknown" image as one of these prototypes. For this purpose, consider each tank described by two rectangular regions, as shown in the Figure above, and use the image registration approach described in section 6.4.2. As an alternative to the area of each region use the ratio l/w , where l and w are respectively the length and width of a rectangular region.

- 6.23 Consider the prototype drawing of Figure 6.23. Use the *Hopfield* program in order to determine the compatible assignments with the line drawings in the Figure below.



- 6.24 Use the *Hopfield* program in order to determine the homomorphisms for the graphs shown in Figure 6.28. Compare the results obtained with those of Table 6.7.